



ALM & Scrum

Necessary But Not Sufficient for Agility

I am a firm believer in the people. If given the truth, they can be depended upon to meet any [software development] crisis. The great point is to bring them the real facts.

-

Abraham Lincoln

Ken Schwaber

Agenda

- Agility is necessary
- Empiricism and transparency are necessary for Agility
- Scrum is a tool you use to become Agile
- Done is not always transparent
- Agile releases
- ALM provides the tooling for Agility

Agenda

- **Agility is necessary**
- Empiricism and transparency are necessary for Agility
- Scrum is a tool you use to become Agile
- Done is not always transparent
- Agile releases
- ALM provides the tooling for Agility

Agility (a·gil·i·ty)

–*noun*

- 1. flexibility, the capacity and capability of rapidly and efficiently adapting to change.*
- 2. ability to take advantage of opportunities and responding to challenges while controlling risk.*

Why is Agility important?

- Complex business organizations
- Complex marketplaces
- Internationalization
- Complex Applications
- Riskier Applications
- Competitive advantage
- **Competitive survival**

Air France flight 447 control panel



Stated reasons organizations want to “be Agile”

- Improved relationship with customers, regaining trust
- Flexibility to turn on a dime
- Improved productivity and quality
- Taking advantage of opportunities
- Early elimination of risk
- Early realization of value
- Always knowing exactly where you are in a development/
deployment cycle
- Easier to make changes
- Elimination of waste
- Lean products that reach market faster and are more targeted
- Increased Return on Investment
- Engaged, empowered workers
- Reduced Total Cost of Ownership

Where Are We With Agility?

- Scrum is in its third decade.
- Modern engineering practices facilitate self-organizing, cross-functional teams.
- Training and coaching programs are in place.
- Scrum is widely used to improve agility.
- Necessary culture change is difficult.

Agenda

- Agility is necessary
- Empiricism and transparency are necessary for Agility
- Scrum is a tool you use to become Agile
- Done is not always transparent
- Agile releases
- ALM provides the tooling for Agility

A thermostat for software development



5 MINS

Purpose: Understand the power of empiricism

You are in charge of keeping a 20 x 40 room in this building at a constant 22C temperature through the day. The room does not have a thermostat.

At the start of the day, 8:00 am, you have to set the heating, air conditioning, venting, and blinds so that they will adjust themselves at the appropriate times to maintain this temperature throughout the day.

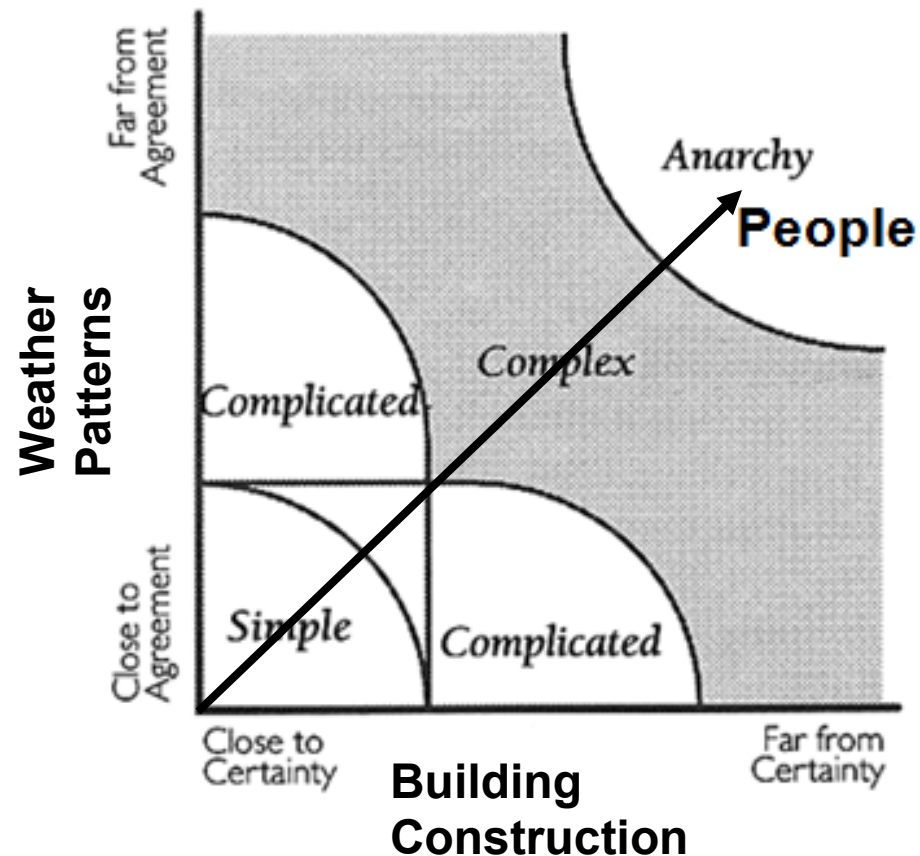
Question: What variables will you have to take into account to know the settings? (hint: number of people in the room will be one variable).

Time	People	Event
7:00 - 8:00	5	Room setup
8:00 - 9:00	50	Breakfast, Continental buffet style, Lyzure Amalgamated
9:00- 10:30	55	Meeting - Lyzure Amalgamated
10:30- 11:00	55	Coffee break
11:00- 12:30	55	Meeting - Lyzure Amalgamated
12:30- 1:00	5-20	Setup and next meeting arriving
1:00- 3:00	50	Meeting - Rexus Ltd.
3:00- 3:30	50	Coffee break
3:30- 5:30	70	Meeting - Rexus Ltd.

Controlling temperature isn't that complex, but there are a lot of things to plan for!

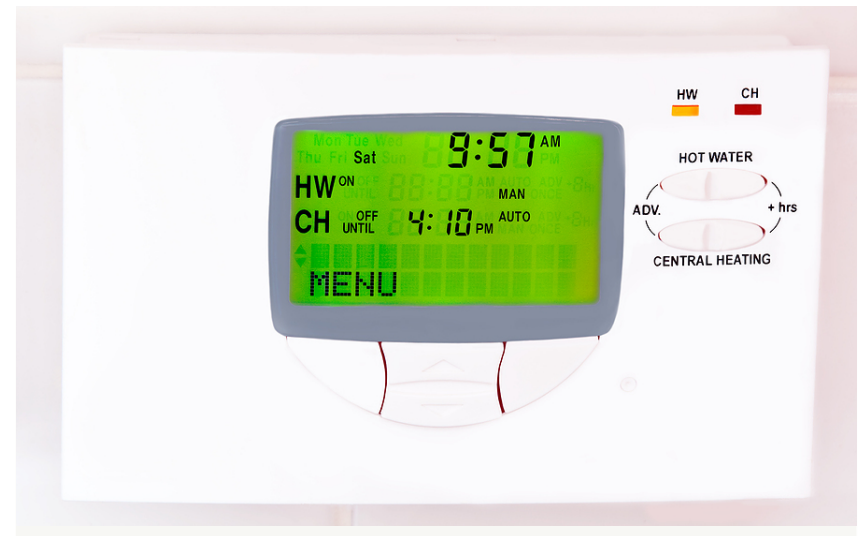
- Variables might include, for any time during the day:
- number of people in room
- metabolism of each person
- activity of each person
- opening/closing of doors
- weather: including sun, clouds, and outside temperature
- temperature of adjoining rooms
- construction material of the building
- floor of the room
- will food be served, when, what type, and how much
- temperature of food brought into room

Dimensions Of Complexity

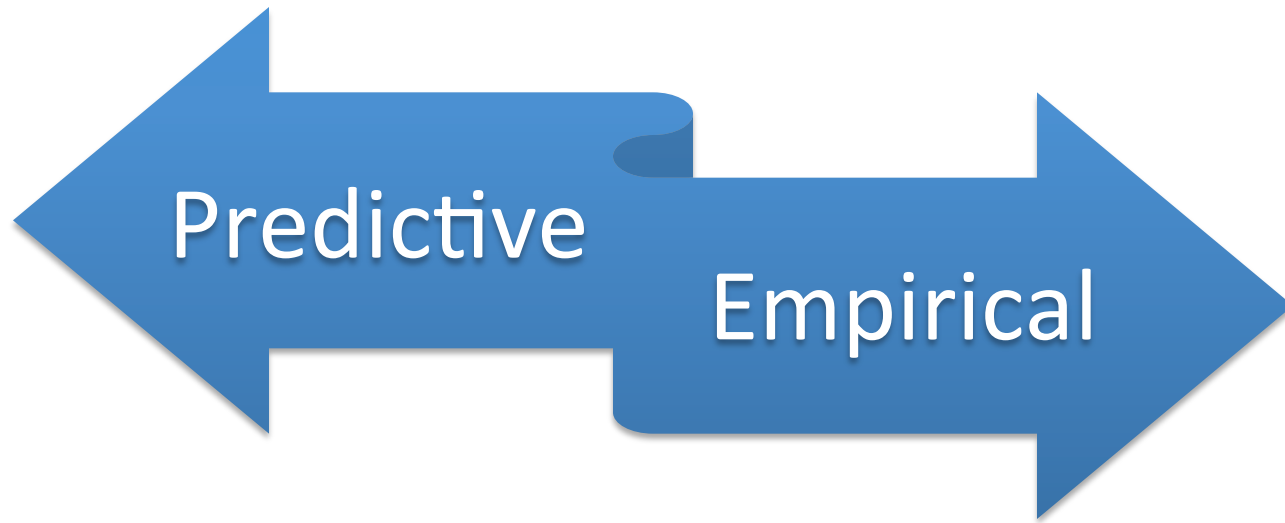


Empirical processes adapt to the future

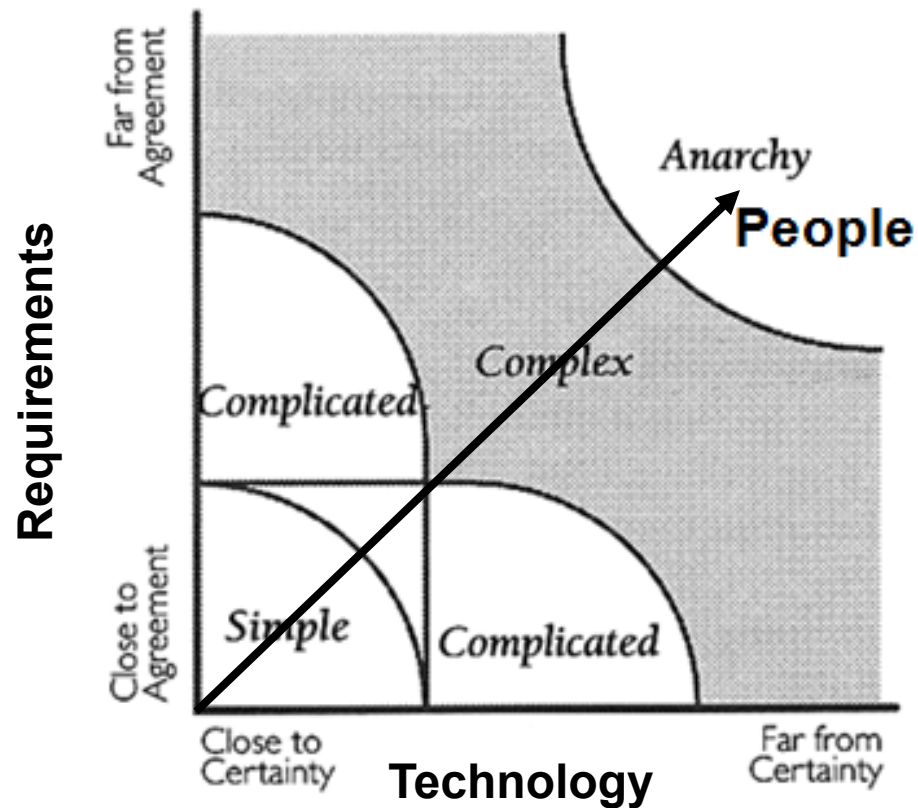
- Variables are ignored. Actual temperature drives setting of air conditioning, heating, blinds.
- Frequent inspection & adaptation (JIT) rather than predictive planning
- Based on “actuals” rather than predictions
- Requires transparency



Two approaches for problems



Dimensions Of Complexity



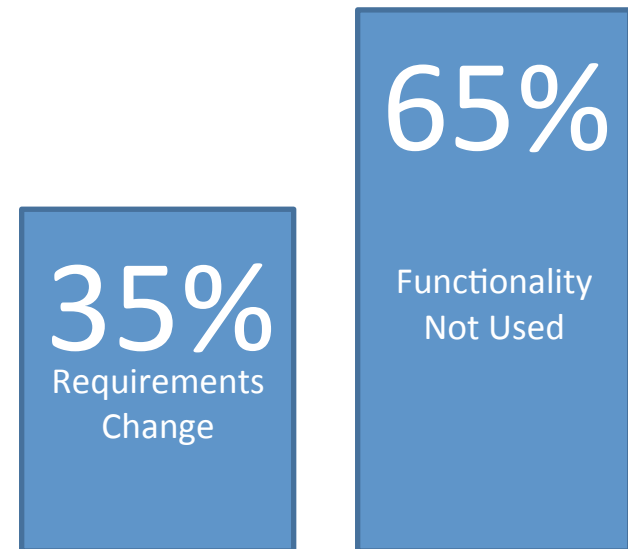
Traditional Product Management



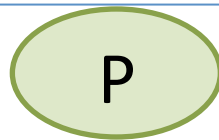
- Multiple roles are created to share the various aspects of the project (Product Marketer, Product Manager, Project Manager)
- Extensive market research, product planning and business analysis
- Product managers are separated from development teams while initial work is completed
- Big up-front product discovery and definition: requirements are detailed and frozen early on
- Requirements are handed over to the development team
- Customer feedback is received late, in market testing after product launch

What that has led to is hundreds of projects that don't realize their full potential, or ultimately fail

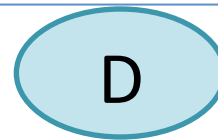
- Based on the premise that the initial information and assumptions are valid throughout the entire planning horizon.
- In defiance of this assumption, the average software project has a 35% change in requirements and around 65% of its functionality, when implemented, is never or rarely used.



Empirical Processes Plan Frequently



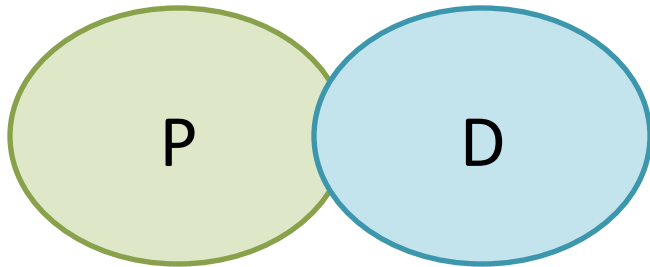
Planning



Doing

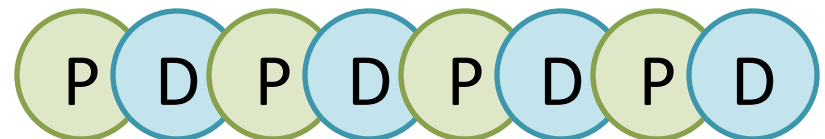
Predictive:

- All planning is done at beginning



Empirical:

- Just-in-time planning and re-planning based on frequent inspection



The first thing empiricism needs is transparency

Transparency (*noun*)

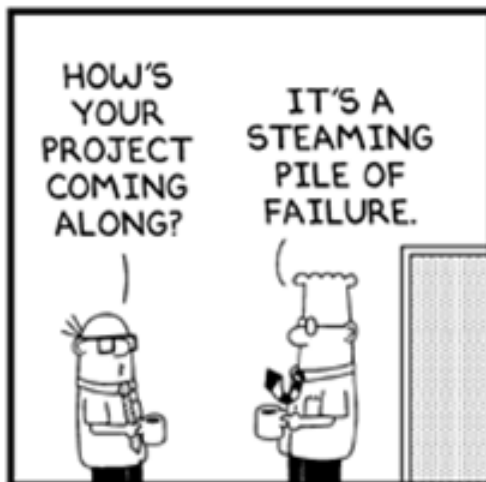
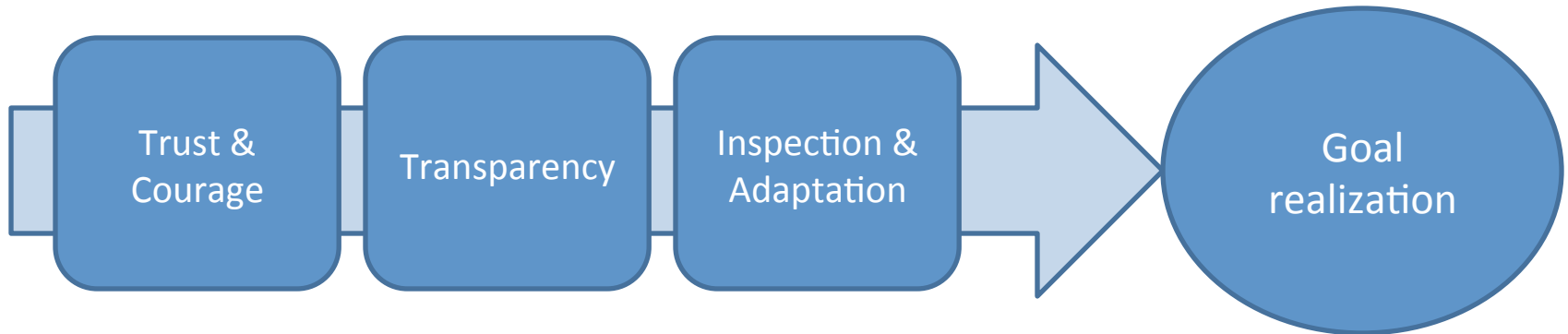
- 1) *Easily seen through, recognized, or understood.*
- 2) *All aspects are equally and commonly understood by all observers*

Most organizations struggle with transparency. It requires trust, courage and a lot of collaboration.

The “Increment”

- Must be “done”, with no work remaining
- Must be potentially usable by customer
- Must be transparent

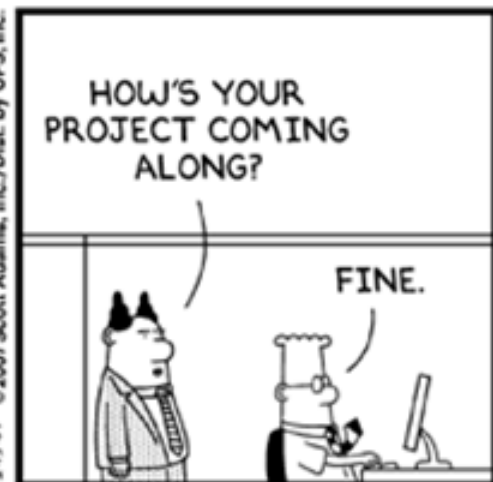
Empirical processes require trust and courage



www.dilbert.com
scottadams@aol.com



3-15-07 © 2007 Scott Adams, Inc./Dist. by UFS, Inc.



© Scott Adams, Inc./Dist. by UFS, Inc.

Exercise (5 Minutes w/3 Ppl Near You)

The Situation:

- You are a developer at xyz co, building life-critical products.
- Your Scrum team is one of seven working on a new release of one product.
- Your team is going to select product backlog to turn into something done (no more work remains, potentially shippable) within a two-week iteration.
- Each team has all the skills to fully develop the requirements into a “done increment.”

The Assignment:

- What work would you have to do to turn the requirements into a “done” increment?
- If you were developing a “done”, potentially shippable increment, what would your definition of “done” be? Would it include, for example, refactoring? What else?

Exercise (Cont.)

Did your definition of “done” include these? Why not?

- Performance testing
- Stability testing
- Refactoring
- Immunological response testing
- Integration with the work of the other six teams
- Integration testing with the work of the other six teams so the increment is the totality of all seven teams
- Release notes
- Internationalization to the six cultures where the product will be sold
- User acceptance testing
- Regression testing
- Code reviews

Quality: A Developer's POV

- I can readily understand the software and where & how things happen;
- When I change or add to part of the software, there are no unintended or poorly designed dependencies;
- I can read the code without looking for tricks or poorly defined and labeled variables or data;
- I don't need the person(s) that wrote the code to explain it to me;
- There are a full set of (automated) tests to check that the function works as expected;
- When I change something and add to the tests, I can check that the entire change and product continues to work;
- How things work and hang together is transparent; and,
- Standard, well-known design principles have been adhered to.

Agenda

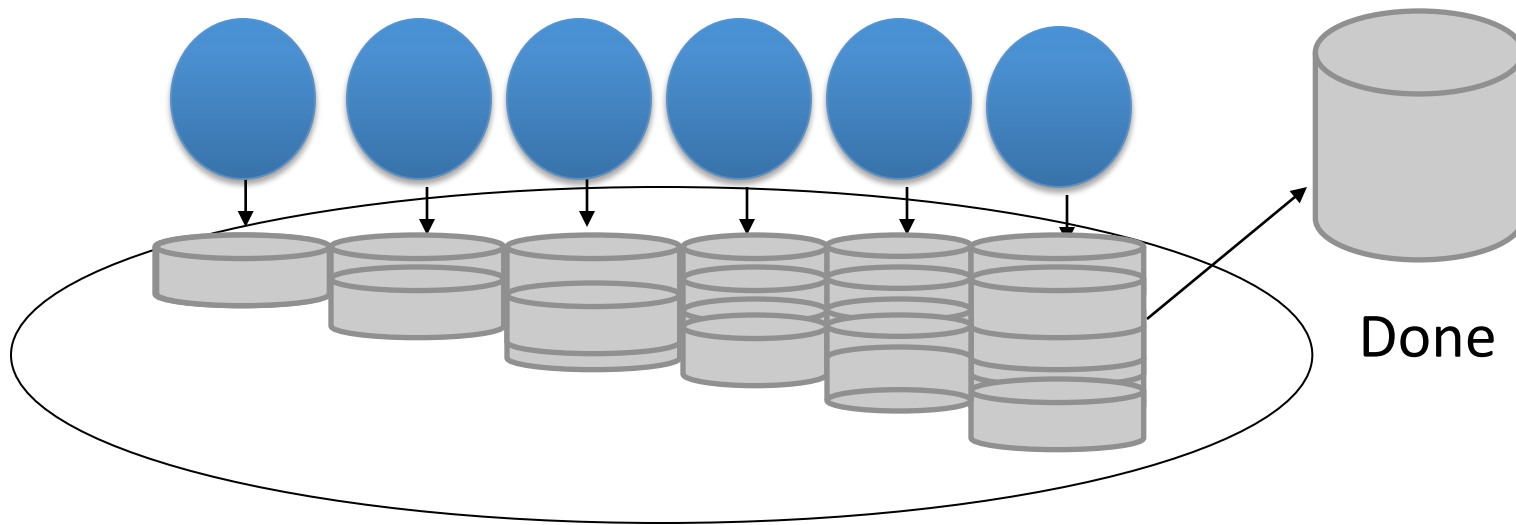
- Agility is necessary
- Empiricism and transparency are necessary for Agility
- **Scrum is a tool you use to become Agile**
- Done is not always transparent
- Agile releases
- ALM provides the tooling for Agility
- Next steps

First, what is Scrum?

Scrum (n): a tool you use
to become Agile

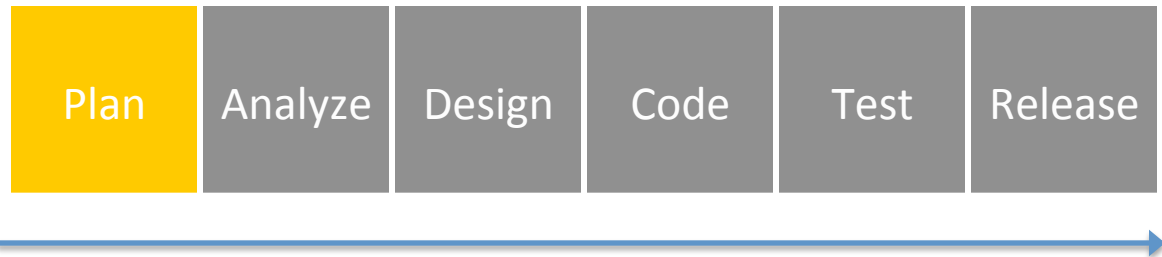


How Does Scrum Work?



Iterative, incremental development with self-organizing, cross-functional teams.

The usual things take a long time, and more ...



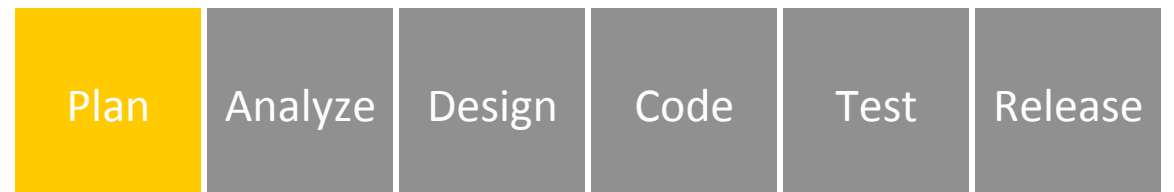
Waterfall

Plan for the entire project up-front, including requirements of all value. Nothing can be used until project is over.

Risk is high, changes are hard to accommodate.

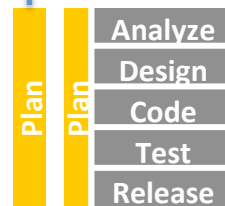
Agile is the usual things done more quickly

Waterfall



Scrum

Short, high value iterations that deliver valuable, opportunistic pieces of functionality.

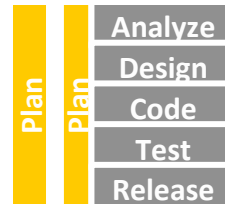


The same work, but processed differently and on fewer requirements.

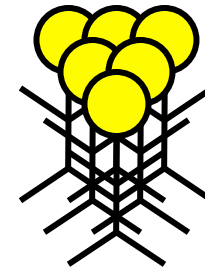
Oh, with I² and self-organizing, x-functional teams

Scrum

Short, high value iterations that deliver valuable, opportunistic pieces of functionality.



Remove Waste!



Scrum Team

Work done by self-organizing, cross-functional teams that are highly productive, creative, and build high quality product.

Low hanging fruit

1. Collocated, self-organizing teams are 100% more productive
2. Don't build low value functionality
3. Don't sustain or maintain low value functionality



Productivity → ROI

- Optimize Return on Investment of each Project (ROI) by increasing productivity

Productivity

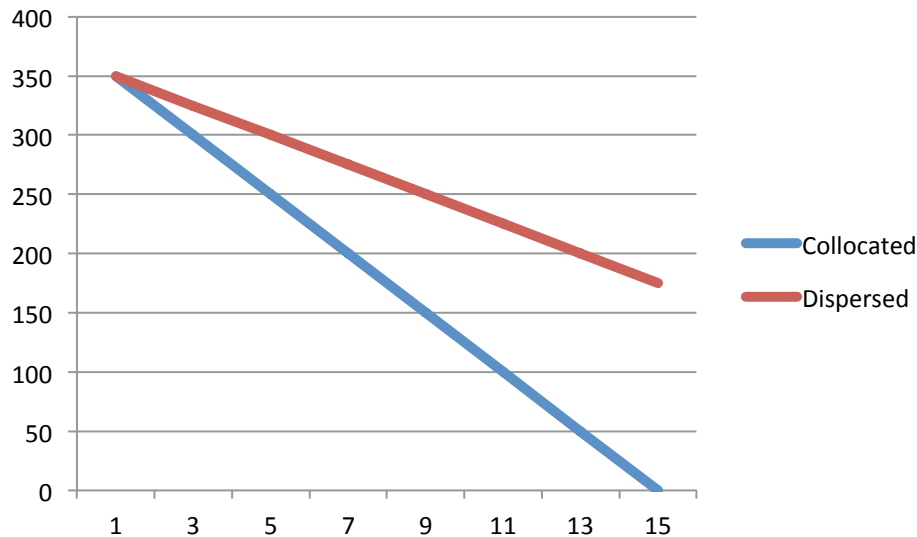
Number of product backlog requirements converted into potentially shippable functionality/\$100,000 investment

Months since Scrum implemented	1	12	24
Productivity	4.5	9.0	12.2
Quality	100+	100	5

Treat People Right To Maximize Productivity

“A collocated, self-organizing team is 100% more efficient than otherwise.”

Source: Boston Consulting Group, 2005



- Scrum does not require team collocation.
- Scrum recommends, but does not mandate team size
- Inspect and adapt

Work On One Thing At A Time

	Team 1	Team 2	Team 3
Projects	Everyone on team is working full time on project A, then B, C, D.	Everyone on team is working 50% on project A and 50% on Project B. When done they start C and D.	Everyone on team is working 25% on Project A, B, C, and D.
Velocity	17	12	4
# of parallel assignments	1	2	4
Total work	307	307	307
Sprints to complete	18.1	25.6	76.8

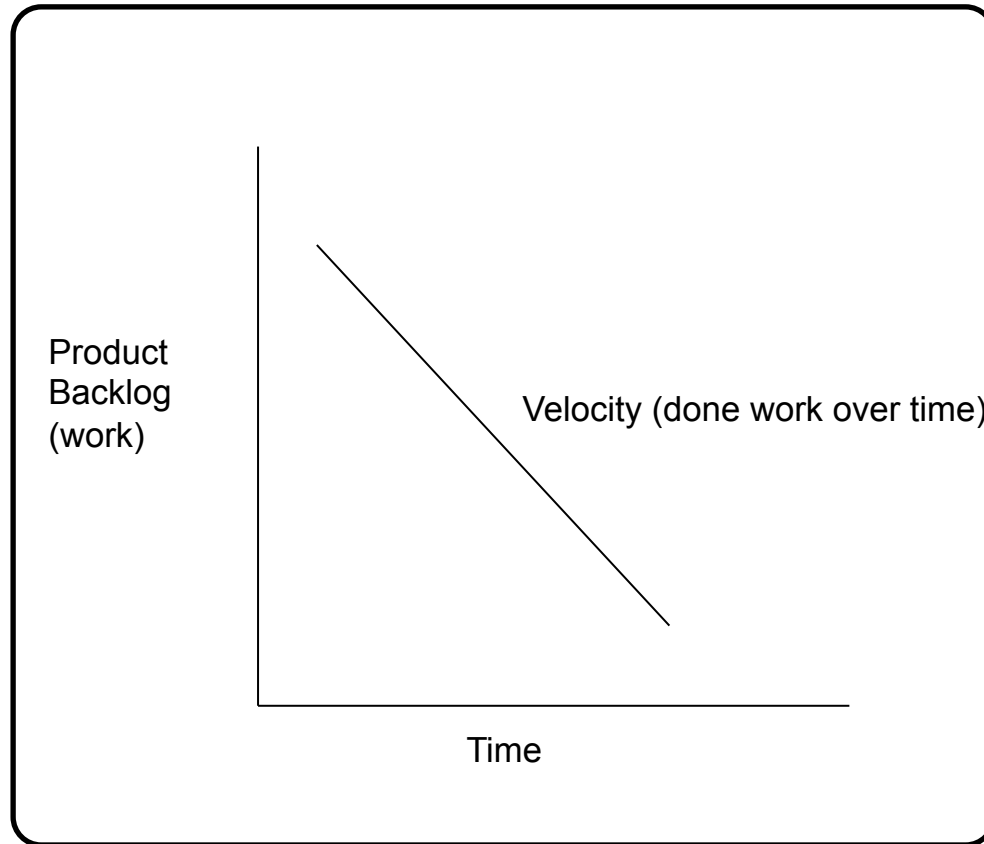
Project A is 62 PBI
Project B is 108 PBI
Project C is 92 PBI
Project D is 45 PBI

When people task switch amongst work, the appearance is of a greater amount of work underway, but the reality is that less work is accomplished. Every piece of work is many times more expensive.

Agenda

- Agility is necessary
- Empiricism and transparency are necessary for Agility
- Scrum is a tool you use to become Agile
- **Done is not always transparent**
- Agile releases
- ALM provides the tooling for Agility
- Next steps

Green Field Software

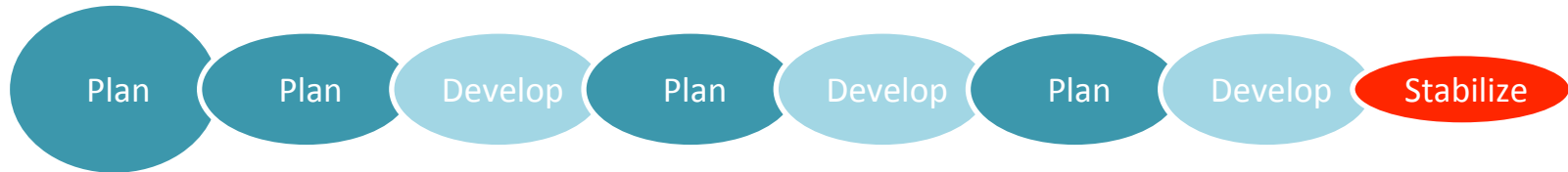


Case Study: TransCanada Pipelines

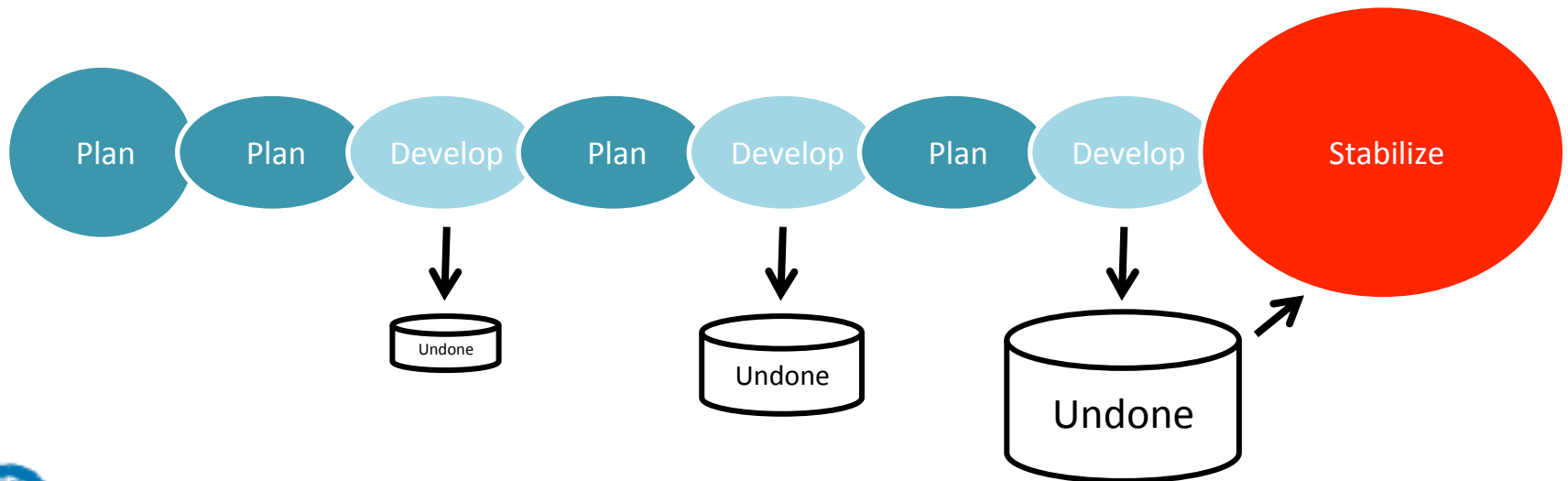
- After three Sprints, Product Owner was delighted. She wanted us to continue, but she wanted us to implement the first three increments so her department could start using them.
- We told her that she couldn't. She asked why not. We said that we weren't done. She said that it looked done. We said, yes, but not that type of done. She asked how long it would take to go from our type of done to the done that would be usable by her.
- Transparency was not present and trust was lost.

Stabilization: What Undone Gets You

Scrum project with complete, integrated “done”:



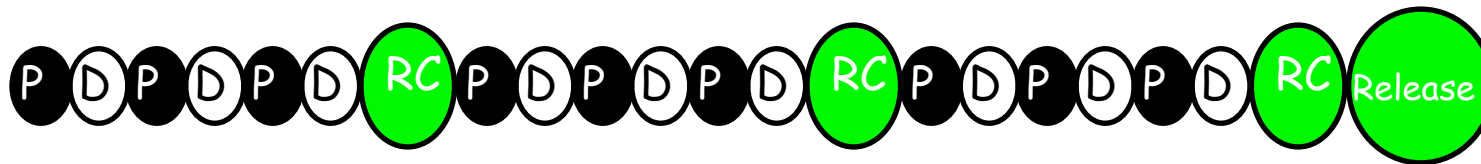
Scrum project with incomplete or variable “done”:



Stabilization: Plan Vs. Reality

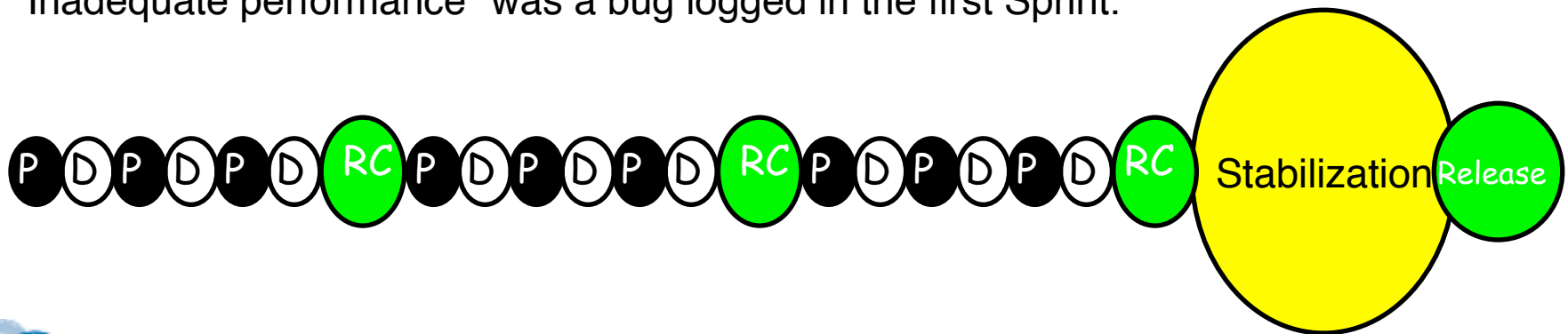
Plan

9 Sprints, 3 release candidates and then release.
800-person development organization.



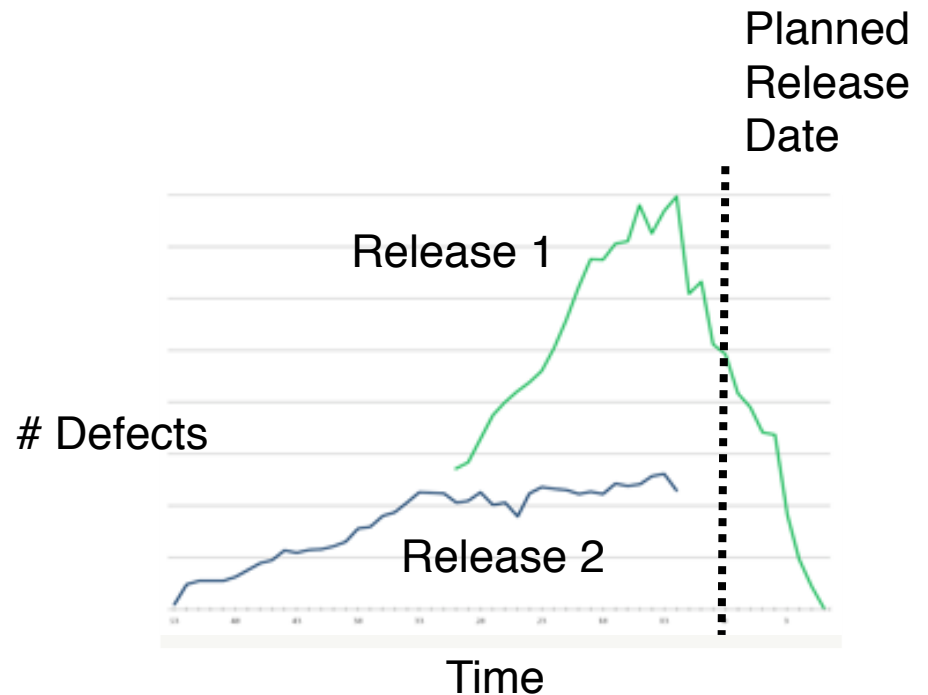
Actual

The release candidates were presentation of partially working functionality from the code branch for each team. A five+ month stabilization was required prior to release. “Inadequate performance” was a bug logged in the first Sprint.

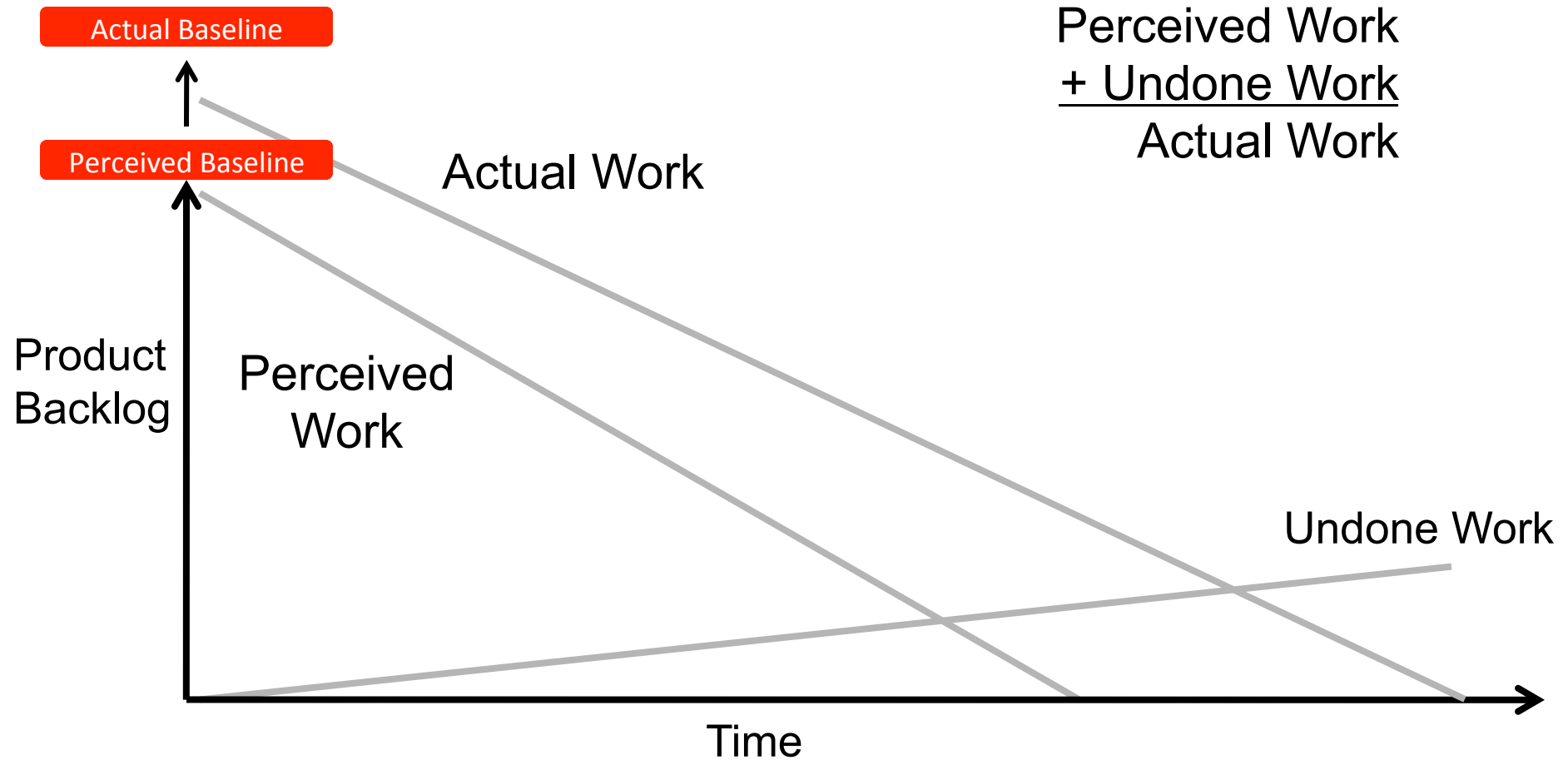


Impact Of Integrated “Done”

- 120 people, 18 teams
- Release 1:
 - Each team produced “done” increments each Sprint, but they were not integrated or integration tested until “code complete.”
- Release 2:
 - All teams produced an increment of integrated, integration tested code every Sprint.



New Baseline Of “Undone” Work



Work Item	Usual	Rec. start	Done
Requirements analysis	25	25	25
Design of architectural components (UI, System, Data)	15	15	15
Design review	0	5	5
Design of tests (system, user acceptance, integration)	0	10	10
Design review	0	3	3
Design of documentation	0	2	2
Design Review	0	1	1
Refactoring of existing design	0	0	8
Design of unit tests for new code	0	3	3
Design of unit tests for code to be refactored	0	3	3
Writing new code	10	7	7
Writing refactored code	6	3	3
Code review (or pair programming)	0	4	4
Write functional tests	8	4	4
Write integration tests	0	4	4
Write documentation	4	4	4
Unit test code	0	2	2
Identify and rectify defects	0	2	2
Subsystem/team build	6	2	2
Identify and rectify defects	1	1	1
Unit test for subsystem/team code	0	2	2
Identify and rectify defects	0	2	2
System/integration build	1	1	1
Identify and rectify defects	0	2	2
System, functional tests	1	2	2
Identify and rectify defects	1	2	4
Integration tests	0	0	2
Identify and rectify defects	0	0	5
Performance tests	0	0	1
Identify and rectify defects	0	0	2
Security tests	0	0	1
Identify and rectify defects	0	0	2
Regression test	0	2	2
Identify and rectify defects	0	8	8
Documentation test	0	1	2
Identify and rectify defects	0	1	1
Total work expended requirement	78	118	148
Work remaining per requirement	65	30	0

Exercise: The Art Of The Possible (5 minutes)

- Purpose: To explore the impact of different approaches to problem solving.
- Explore the difference between planning a party if every sentence begins with “yes, but” and “yes, and.”

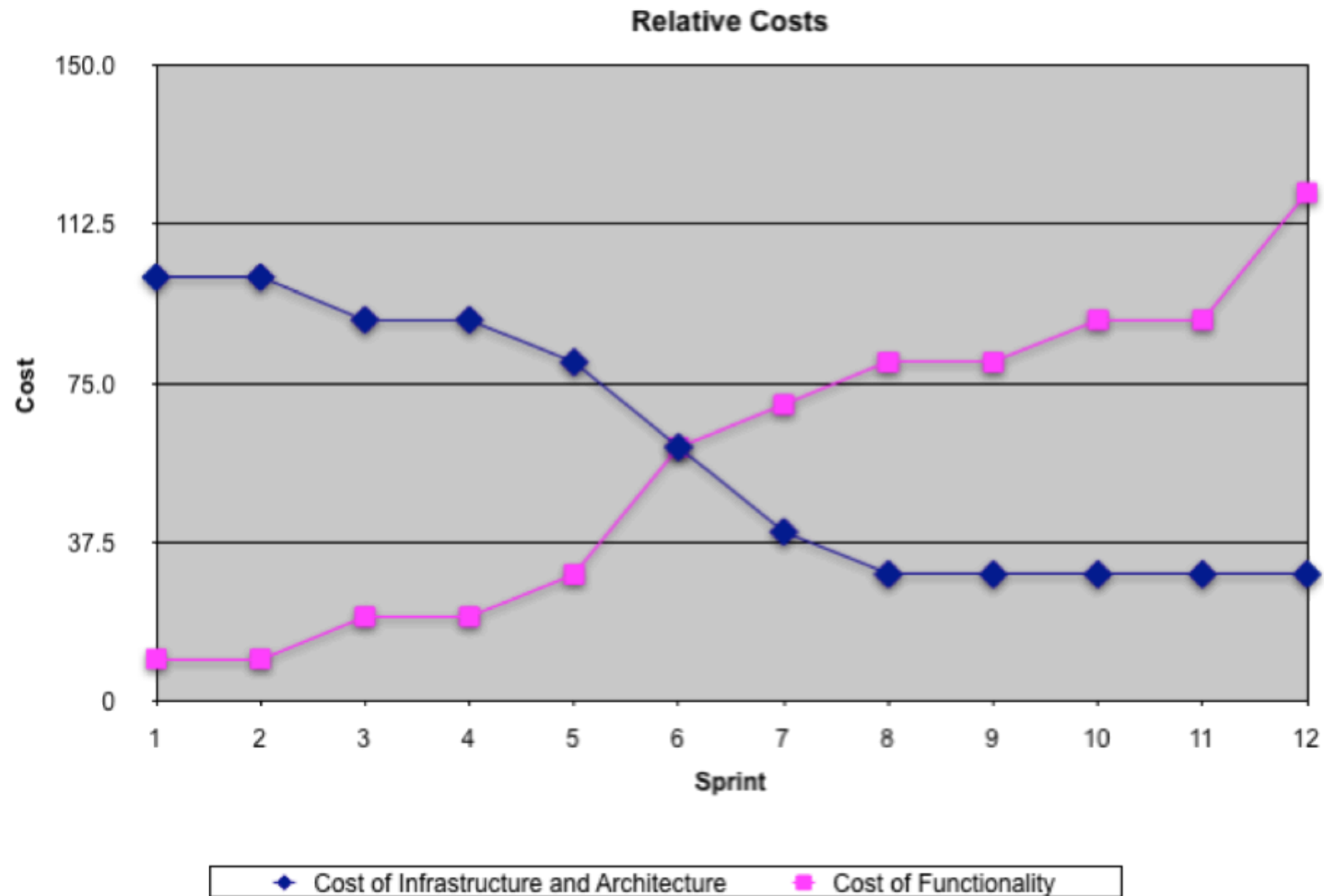
Architecture Was A “Yes But”

- We are used to doing all of the architecture and infrastructure at the beginning of the project ... to make sure that everything is in place when the developers get going.
- Our best people are used for this work to ensure it is excellent.
- This often builds architectures and infrastructures appropriate for the initial requirements, but difficult as changes are required and excessive for the value delivered.

Emergent Architecture Development

- Architecture and infrastructure are high priority non-functional requirements or the requirements to fulfill them are included in the definition of “done.”
- Must be completed to prove that functional requirements can be implemented satisfactorily
- Every Sprint still must deliver at least some piece of business functionality
 - To prove that architecture or infrastructure works
 - To prove to customer that work they care about is taking place
 - Basis for estimating

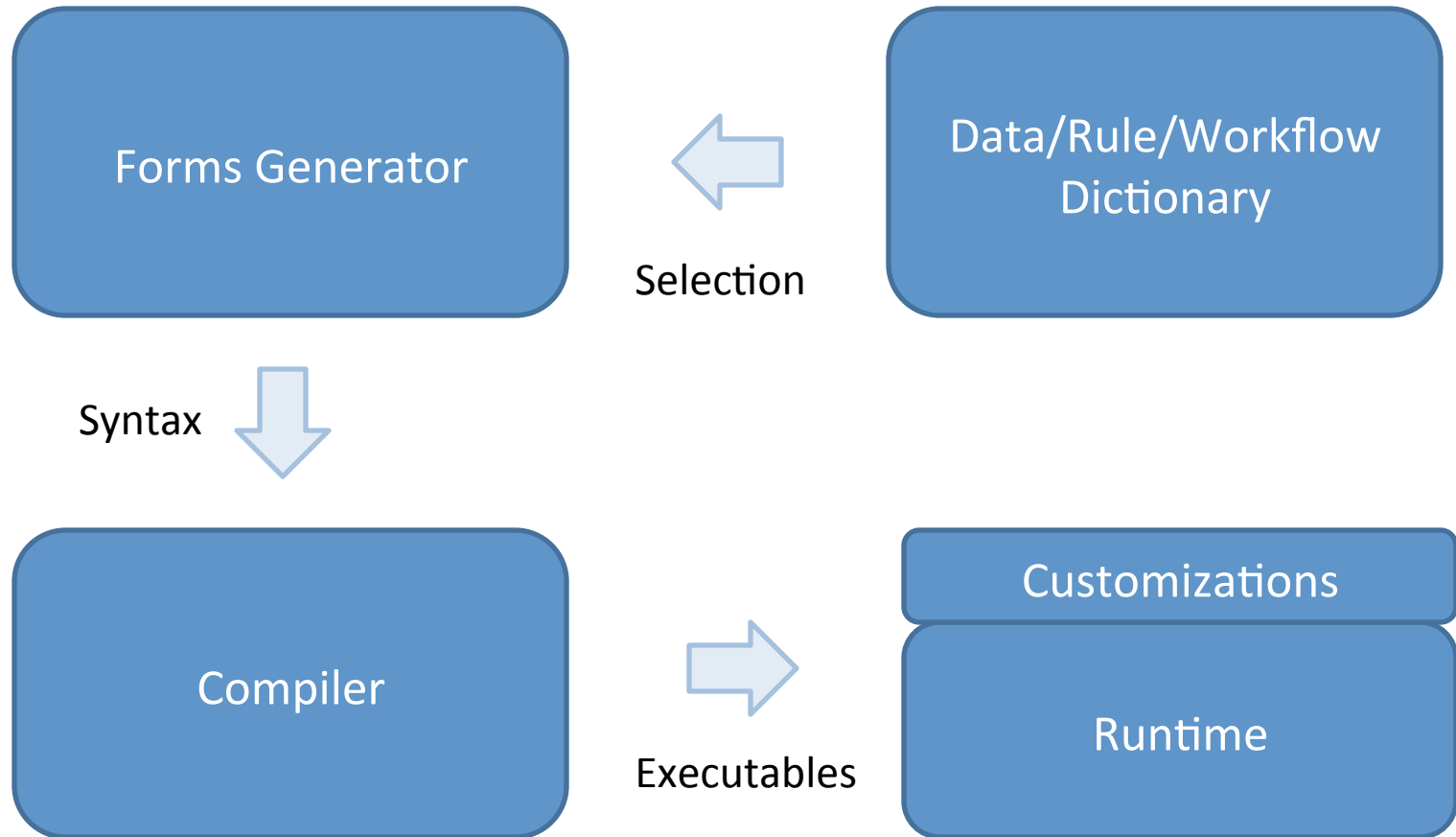
Architecture Changes -> Cost Changes



Primavera Case Study

- Market opportunity perceived in expanding the footprint through adding collaboration and workflow capabilities.
- Primavera partnered with iManage.
- Eleven month project, 91 things had to be done to implement collaboration and workflow. Use reliable waterfall.
- Team of 10 best formed to figure out how to bolt two products together.
- Two products had different security systems and design architecture. Three months of struggle.
- Shift to Scrum. What can be done in one month?
- What can be done in second month from prioritized list?
- At the end of seven months product shipped, 62 requirements done.
- Next release used none of remaining 29 requirements.

TurboTax Product Family Architecture



The Art Of The Possible At Intuit

Problem: Tax preparers need to be able to add customer questions to different forms so that they can serve their customers more specifically.

Initial Estimate: Development estimated functionality available within 2-3 years.

Agile Solution: Drive One Requirement Through Architecture to determine feasibility and create estimates. Then let solution and functionality emerge.

First Product Backlog Item: Tax preparers need to be able to determine if the demographic of the taxpayers zip code indicates that the number of dependents is excessive on form 1040-a32.

Decompose Product Backlog 1.a so it consists of items of 16 days or less that can be “done” in one monthly Sprint.

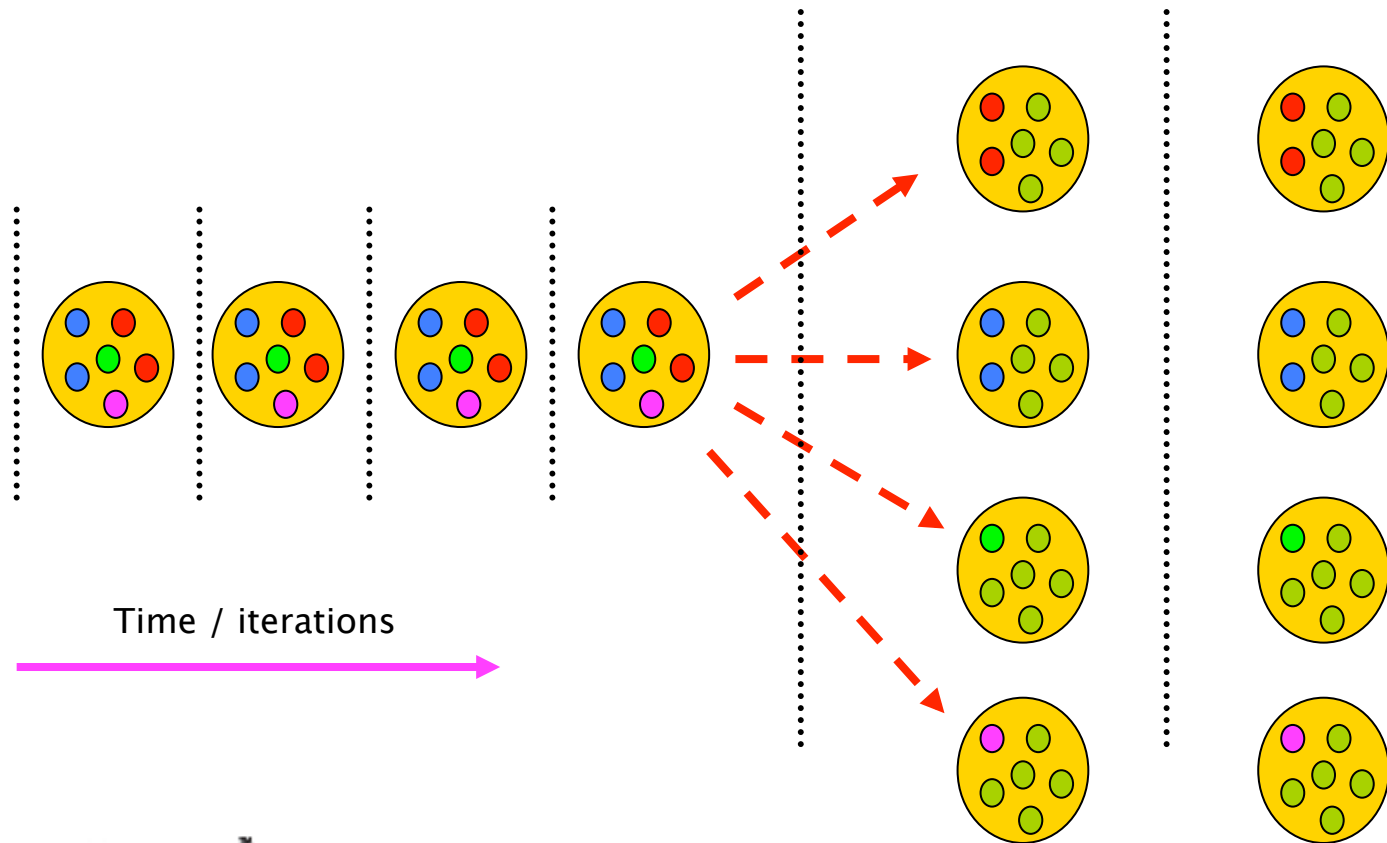
Effort Team

Product Backlog Item

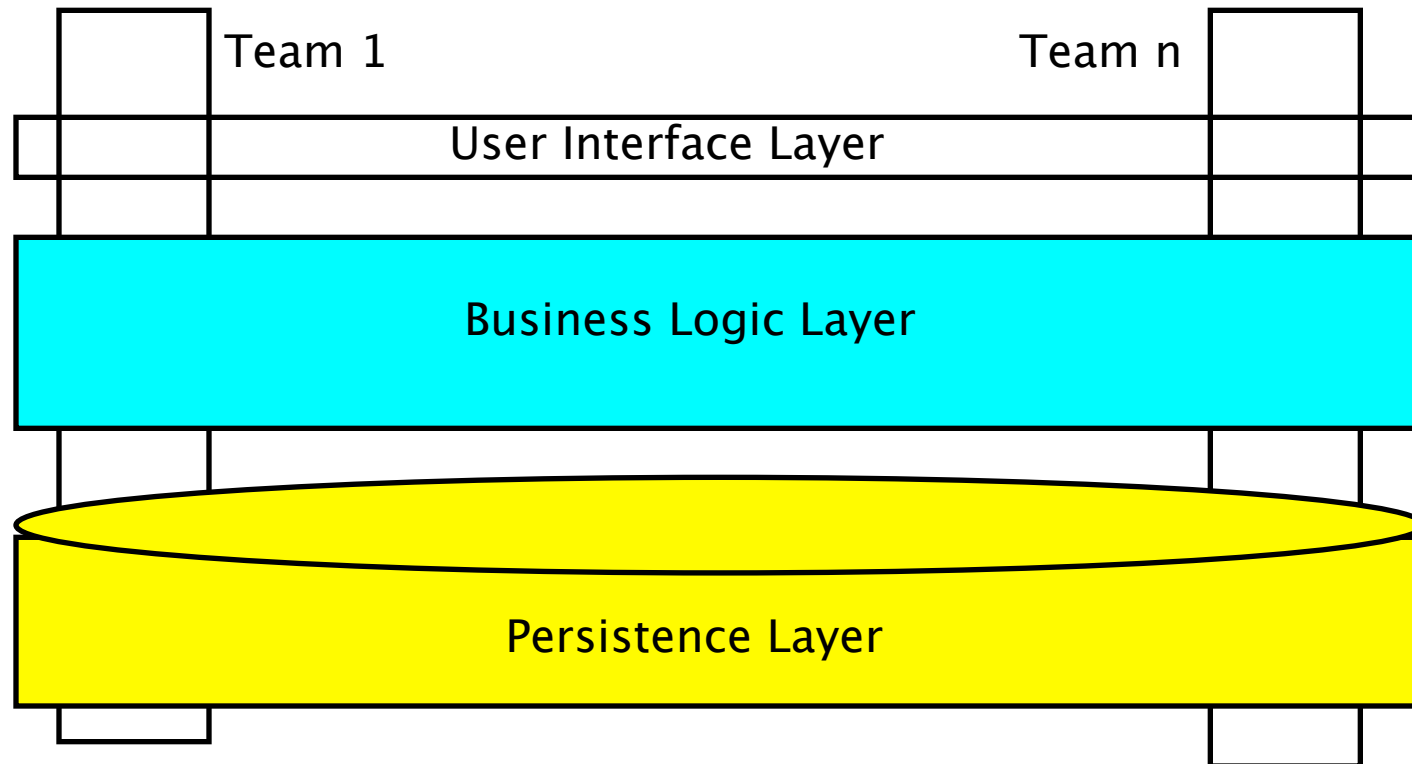
- 72 CTP Tax Preparer can enter rule zipcode "24760" and demographic "CA" and dependents >10 on form 1040C-a2 so that it causes an exception
- 16 CTP Tax Preparer can enter rule zipcode "24760" and demographic "CA" and dependents >10 on form 1040C-a2 into data/rule/workflow engine so that it is entered into the data dictionary
- 6 CTP Data selection engine can select form 1040C-a2 from the data/rule/workflow engine so that it also extracts custom rule zipcode "24760" and demographic "CA" and dependents >10
- 16 CTP The forms generator can generate syntax for form 1040c-a2 so that rule zipcode "24760" and demographic "CA" and dependents >10 is syntactically correct and recognized by the compiler
- 16 CTP Compiler can recognize and act on syntax for form 1050c-a2 with rule zipcode "24760" and demographic "CA" and dependents >10 on form 1040C-a2 so as to generate the appropriate run time code for the runtime engine
- 12 CTP The run time engine recognizes and binds the executable so that when form 1040C-a2 is reached the the rule zipcode "24760" and demographic "CA" and dependents >10 can be located at the proper time and executed
- 6 CTP Tax Preparer can enter rule zipcode "24760" and demographic "CA" and dependents >10 on form 1040C-a2 so that it causes an exception

Start With A Small Team Then Grow

- Start with a small tiger team to establish the core.
- Then move the members to start new teams

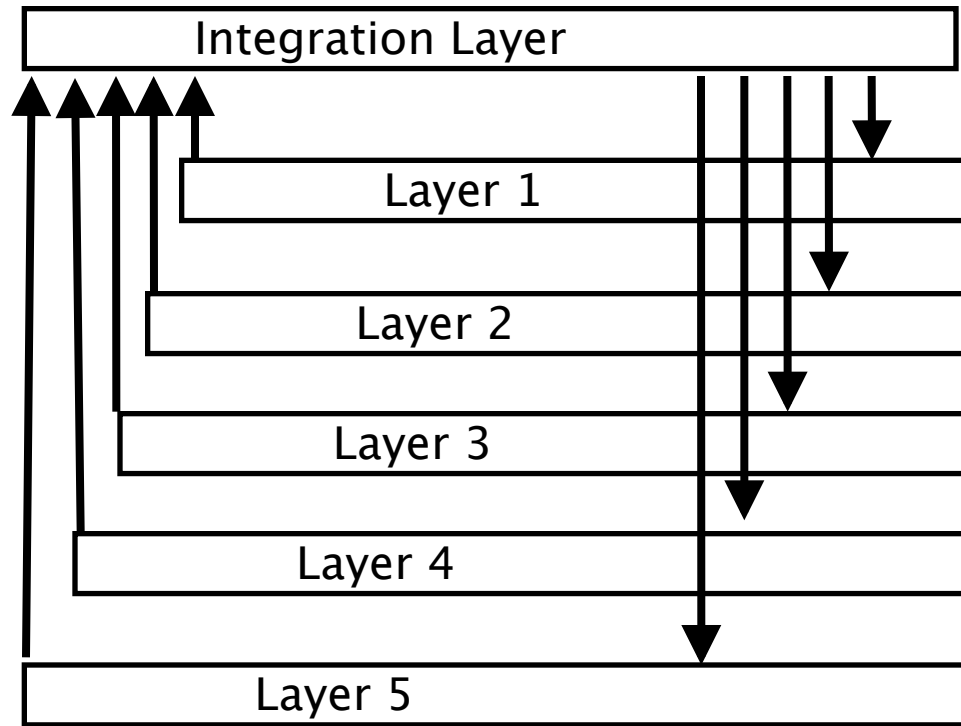


Functional Integration Of Teams



One code base, continuous integration, work divided by functionality

“Layer” Integration Of Teams



One code base, continuous integration, work tested at integration layer and integration bugs passed back for immediate fixing

ROI Of Software Products

- Traditional software accounting only takes the cost of development into account.
- A more balanced method would take the long term costs and value of the product into account.
- We should also take into account the impact of short term decisions on long term ROI

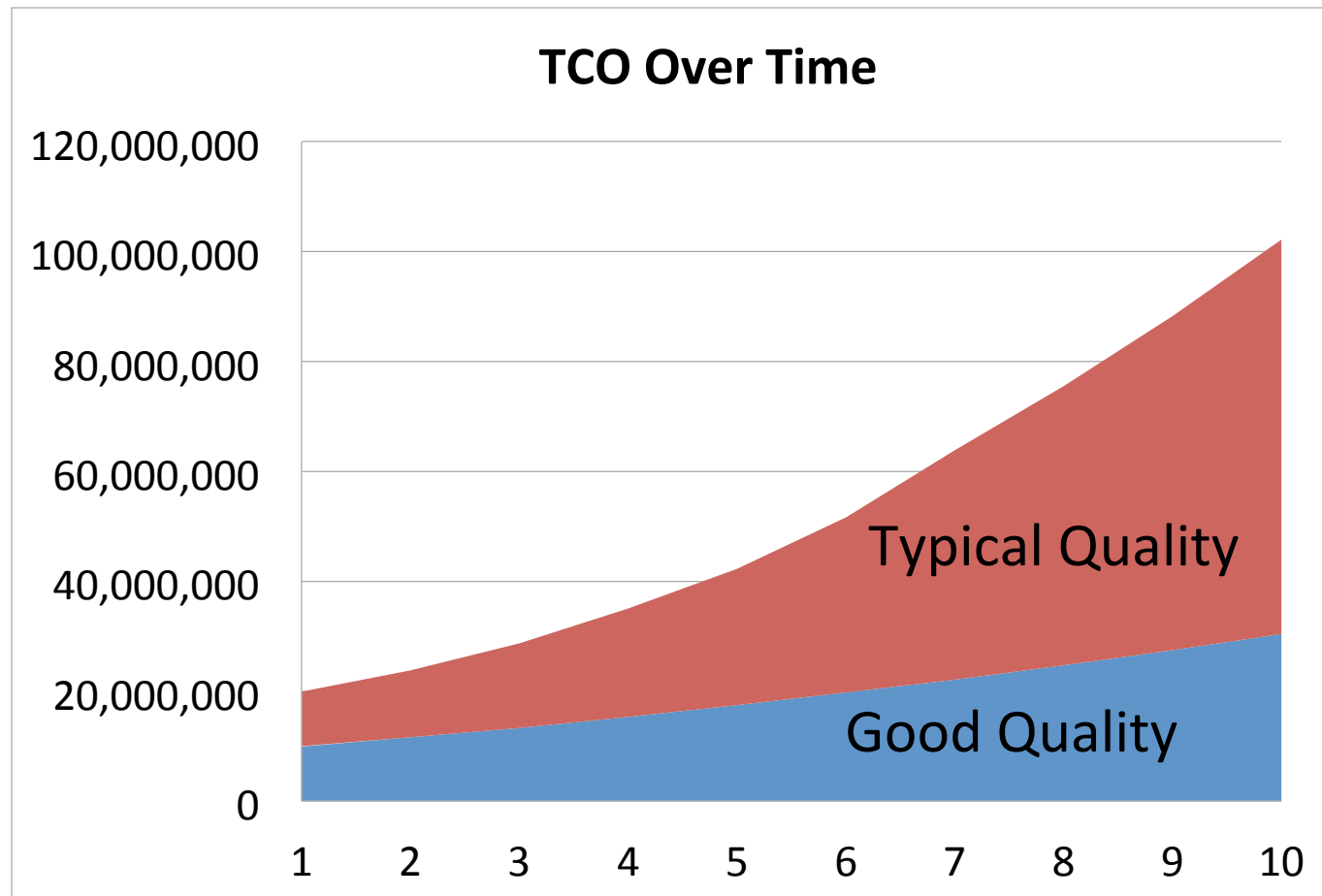


Lifetime Product ROI Variables

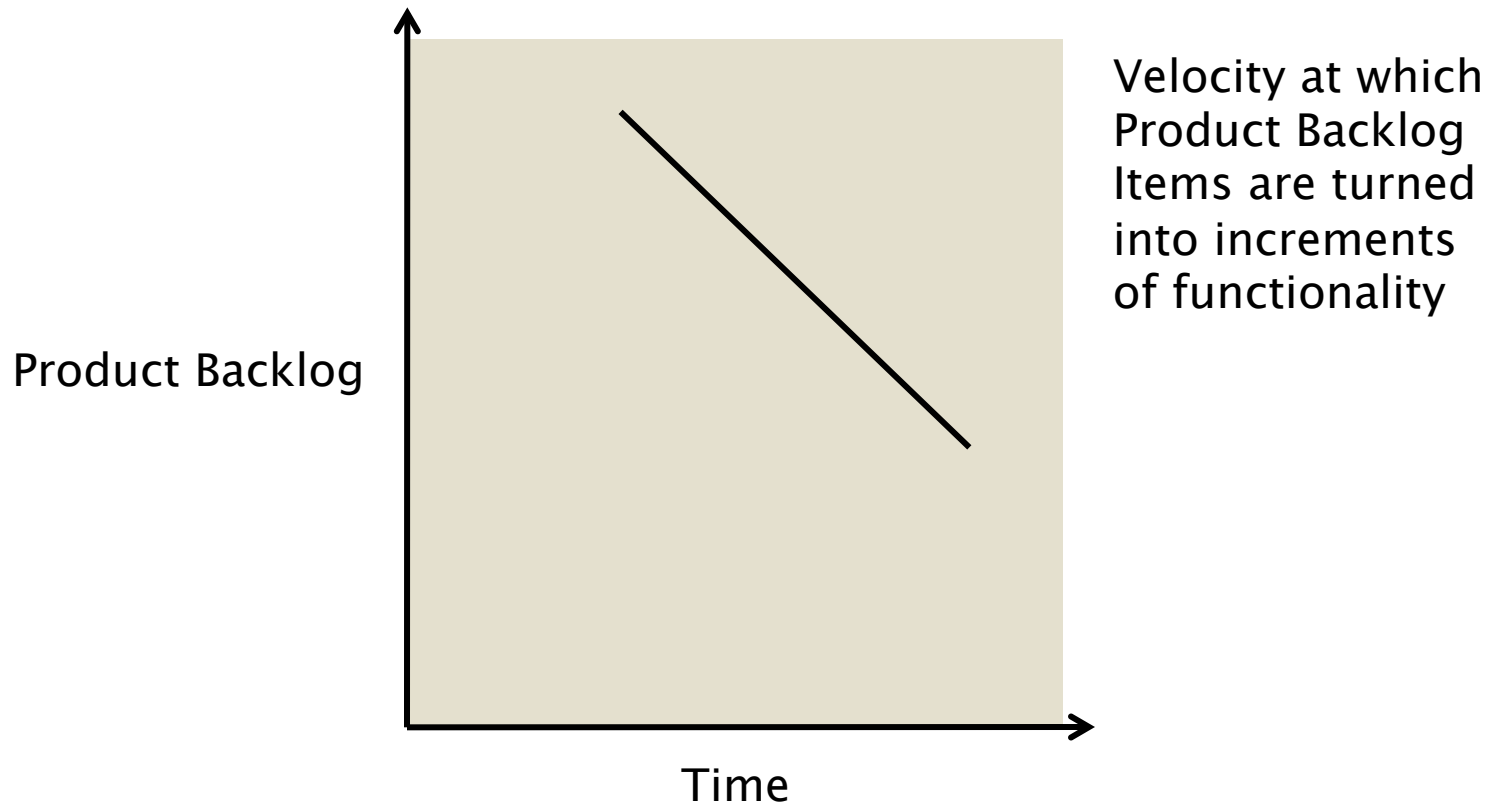
Product ROI over the life of a product is dependent on the variables:

- Presence of valuable functionality for customers to use;
- Absence of low value functionality that must be maintained and sustained regardless;
- Quality code base to predictably maintain and sustain;
- Quality code base to enhance without undue effort or risk;
- Ability to catch defects early and no later than at release;
- Predictability of schedules;
- Lack of surprises; and,
- People and resources available to work on it.

Do The Right Things The Right Way

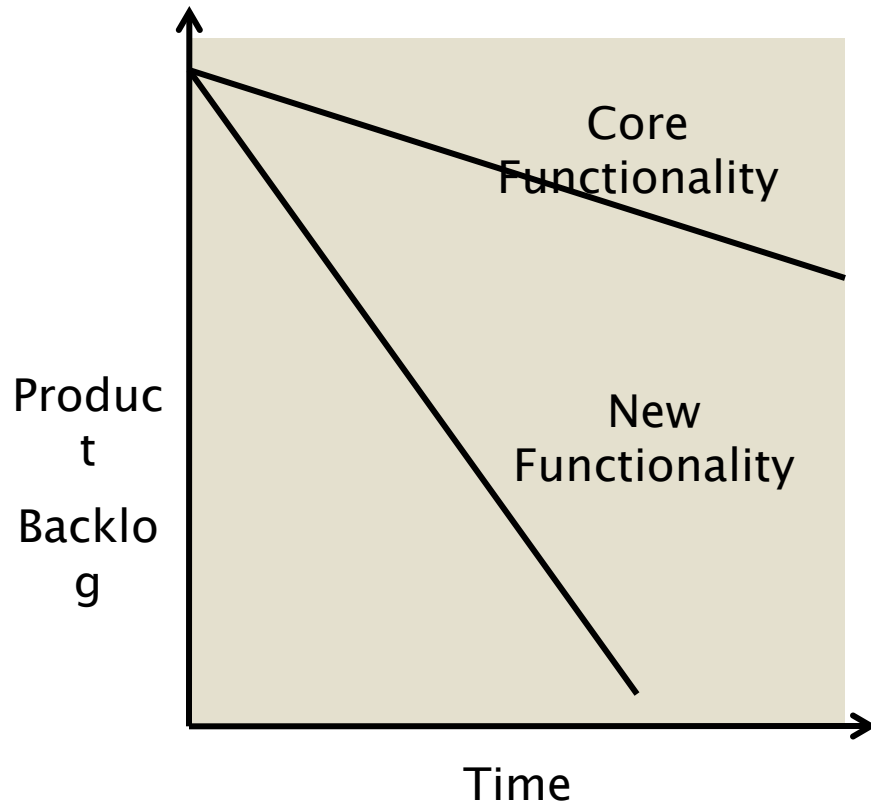


Management Variables



Other variables: Quality, Value

Software W/Low Value To Developers

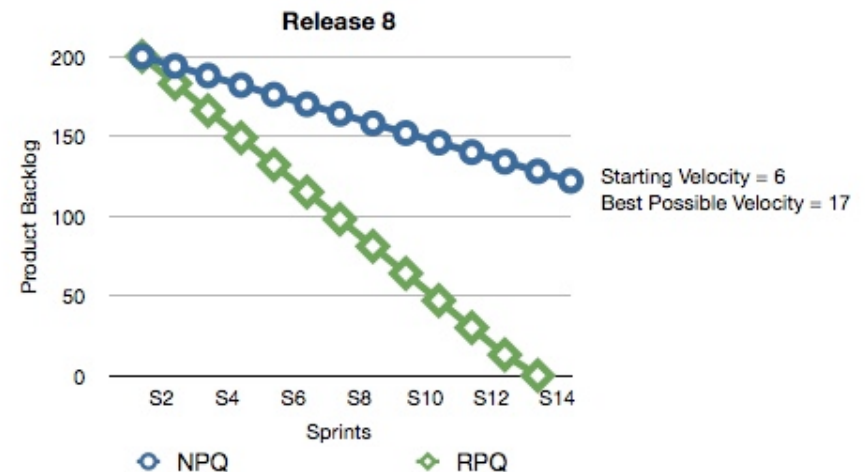
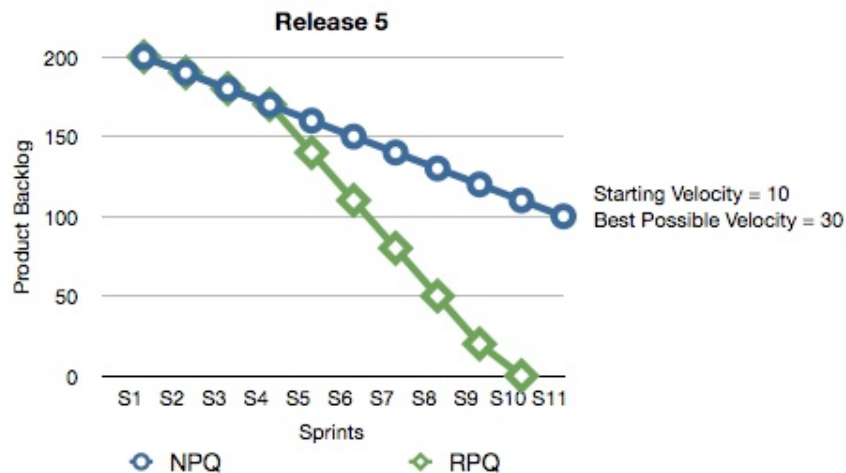
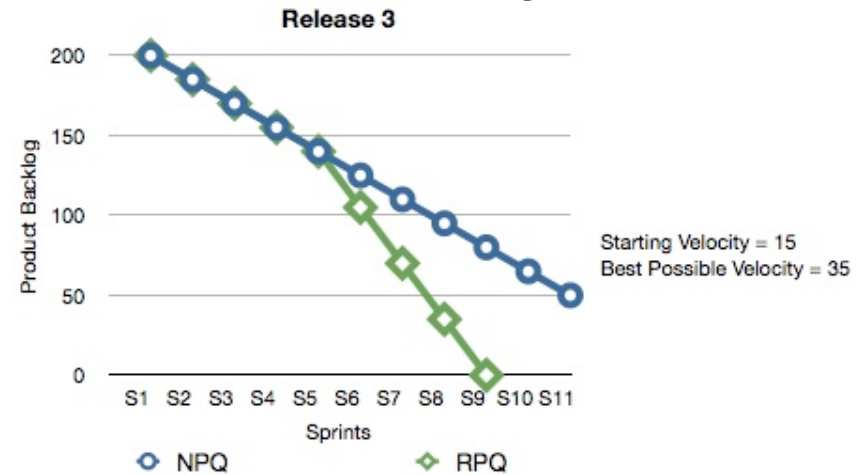
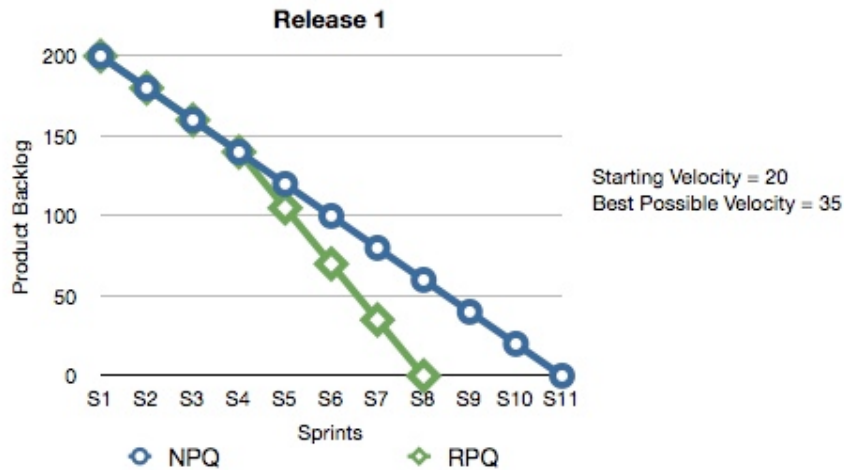


Core Functionality

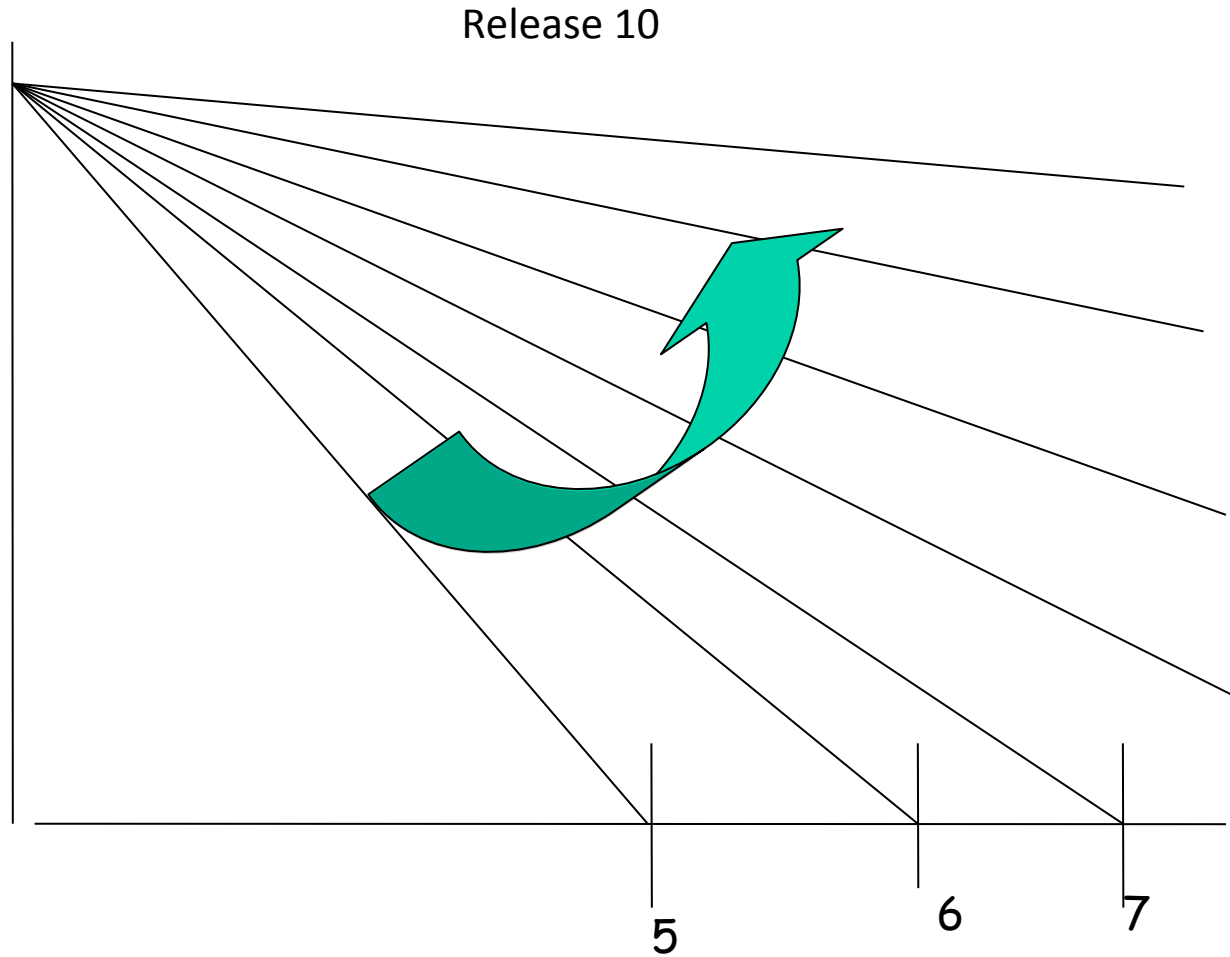
- Most significant new functionality builds on it;
- Also called infrastructure and legacy software;
- Is fragile, doesn't have complete test harnesses, and few people still know how to or are willing to touch it; and,
- Requires more time to work on; velocity working on it is less than new work.

Gradual Impact Of Reduced Quality

5



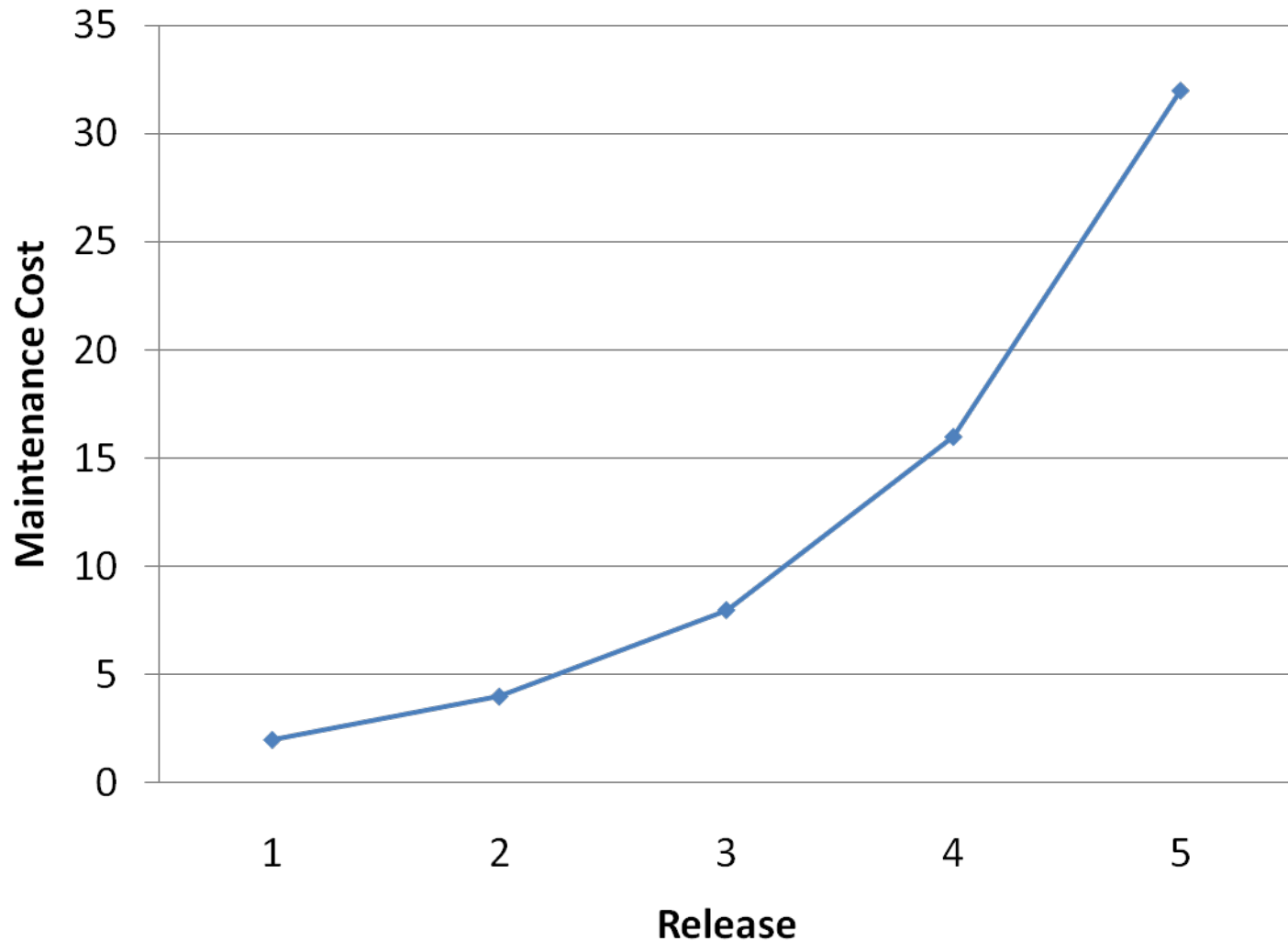
Reduced Velocity Puts Organization In Corner₁₉₂



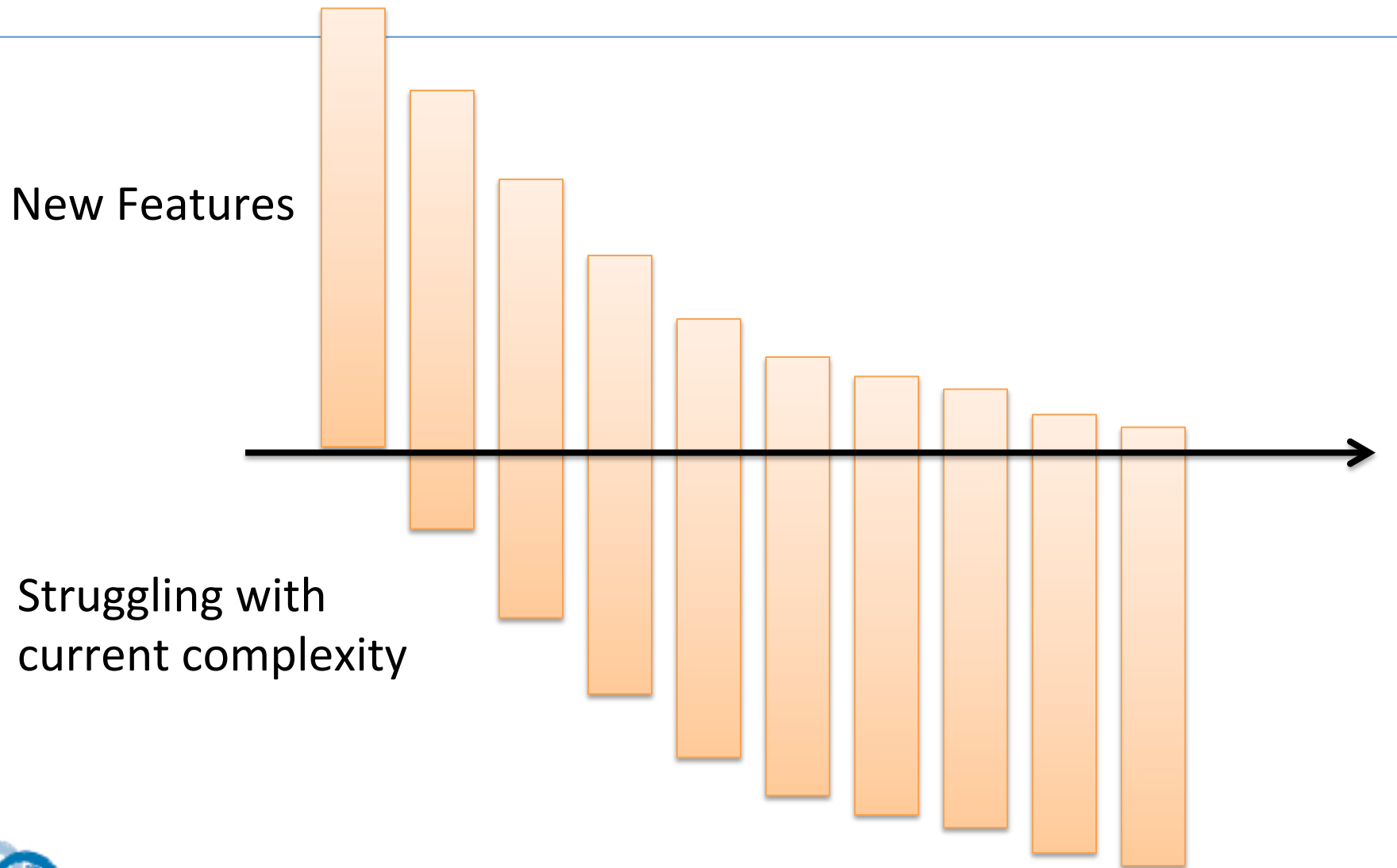
Exercise (5 minutes)

- Your CEO comes to your development group. She tells you it is mandatory to deliver three new pieces of functionality within three Sprints. Otherwise, competitors will decimate the company.
- Planned work consists of:
 - Function 1: 20 units of work, 15 new, 5 core
 - Function 2: 40 units of work, 25 new, 15 core
 - Function 3: 30 units of work, 20 new, 10 core
- Velocity for new functionality is 15 units of work per Sprint per team.
- Velocity for core functionality is 5 units of work per Sprint total.
- You need a release with all three functions in three months.
- Can you do all three functions in 3 months? Can you save the company?

How Does Design-Dead Software Smell?



Technical Debt



Forms of Technical Debt

- Defects
- Lack of automated build
- High code complexity
- Lack of automated deployment
- Lack of unit tests
- Highly coupled code
- Business Logic in the wrong places

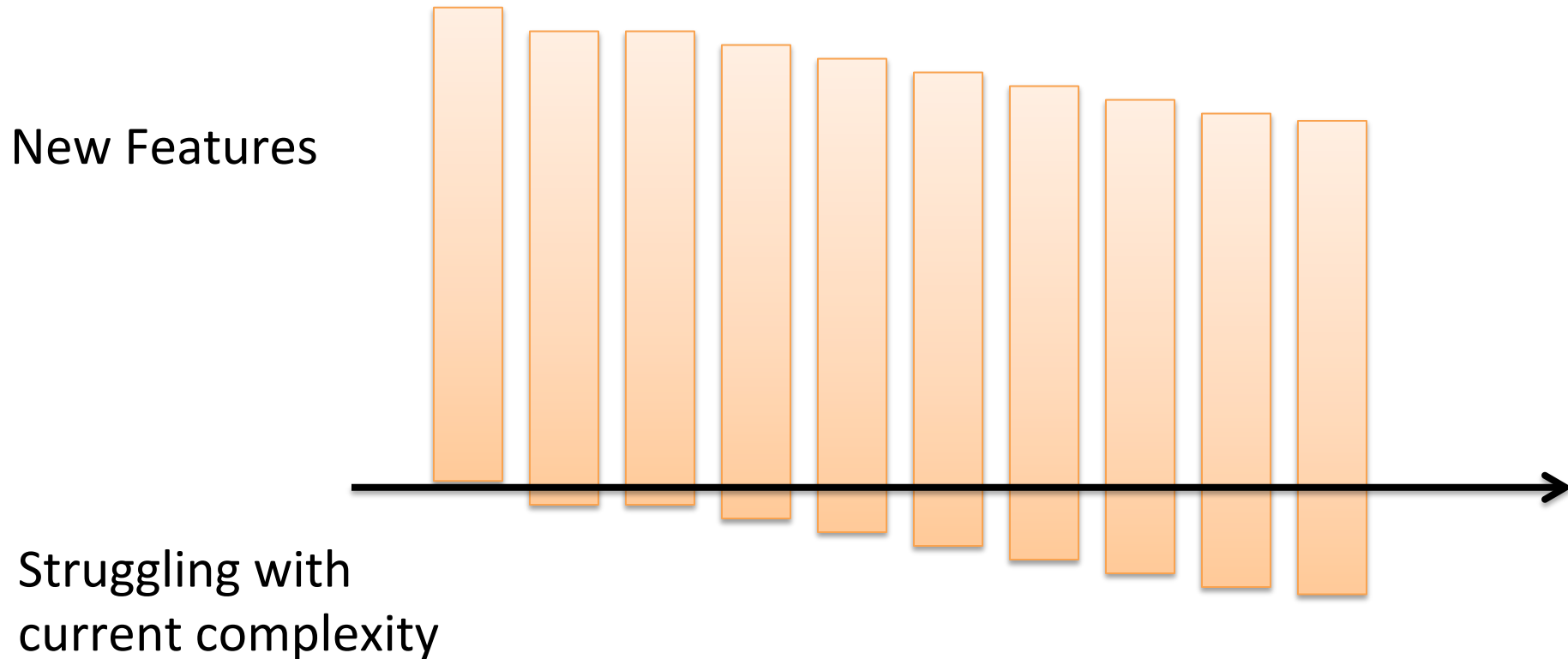
- Lack of acceptance tests
- Inability to regress current functionality
- High cyclomatic complexity
- Duplicated code or modules
- Unreadable names or algorithms

Paying Back Technical Debt

1. Stop creating debt
2. Make a small payment
3. Repeat from 2

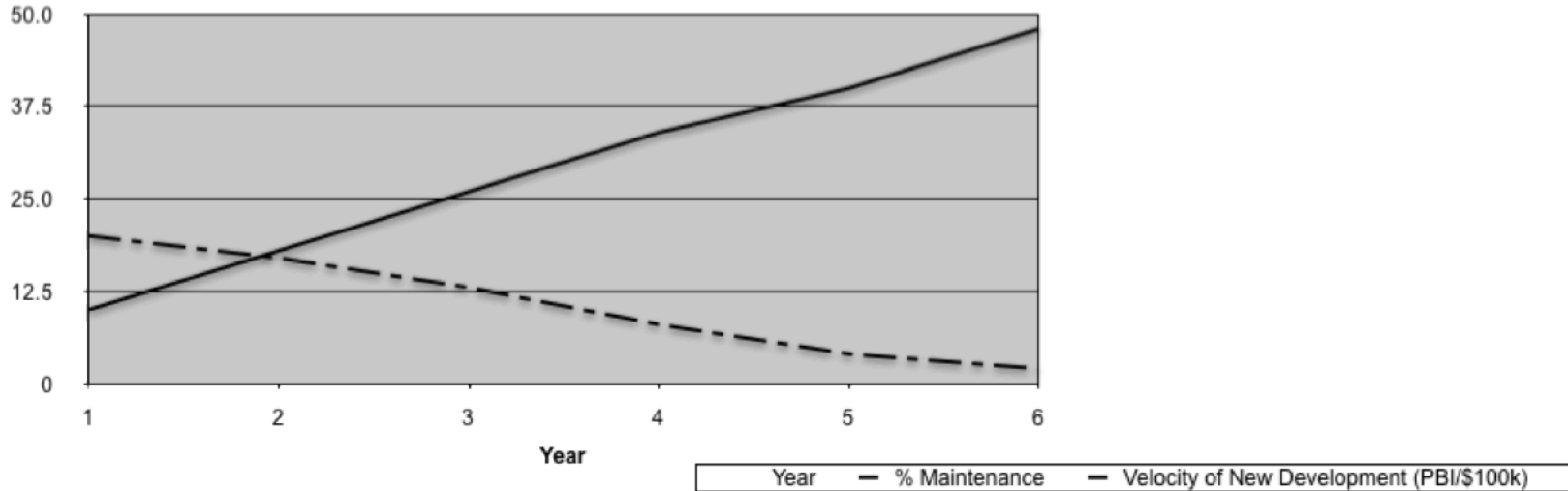


Technical Debt in a Healthy Team

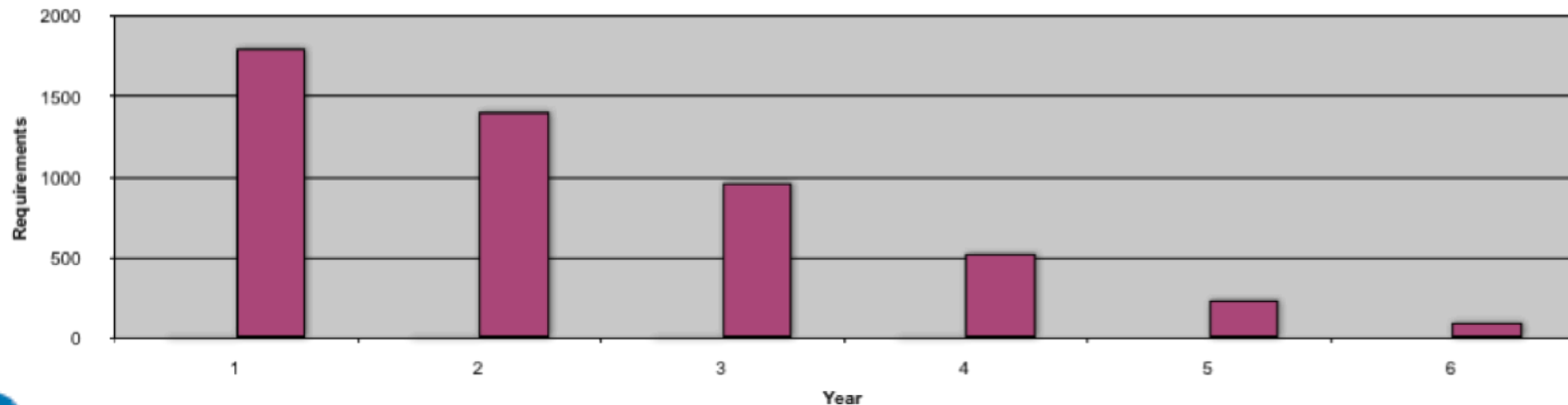


Maintenance/Velocity/Features Correlation

Correlation between declining quality and velocity



New Requirements Capability



Optimize TCO

- The bugs you don't fix reduce the value of the system in increase the cost of maintaining and sustaining it.
- The design and code you don't refactor makes it increasingly difficult to enhance and sustain a system.
- The complete automated test harnesses you do not have mean that you can't tell when the system breaks or what it does.

Agenda

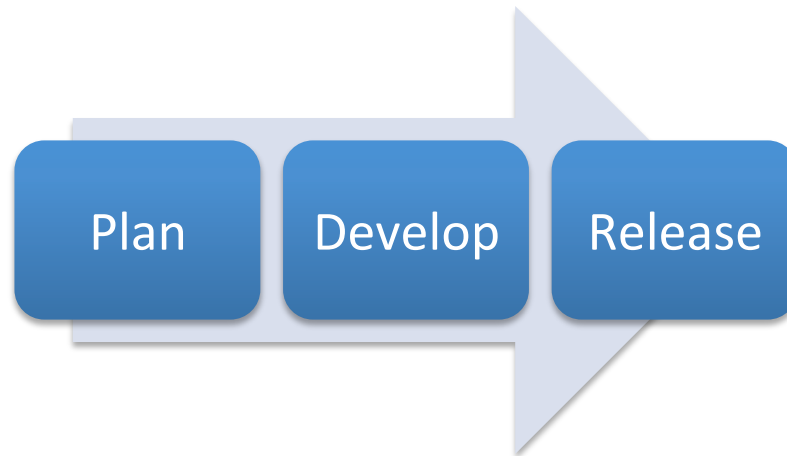
- Agility is necessary
- Empiricism and transparency are necessary for Agility
- Scrum is a tool you use to become Agile
- Done is not always transparent
- **Agile releases**
- ALM provides the tooling for Agility
- Next steps

A release? What a great idea!

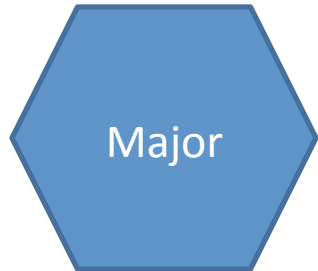
A release sounds like a really good idea. We should do one, do them often, and triumph!

Our primary constraints are:

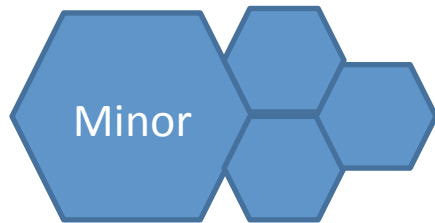
1. The inadequacy of our development environment
2. The miserliness of our and our customer's habits
3. The organization of our product



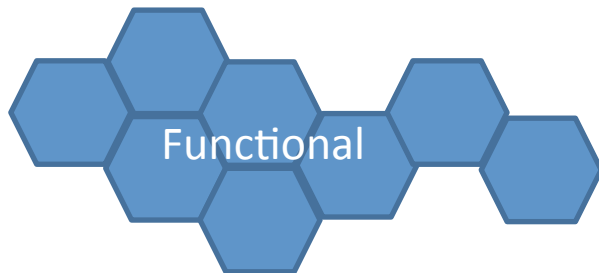
Three primary release strategies



Lots of changes, happen infrequently, freezes other work, relatively stale functionality, high customer absorption costs.



Lots of broad changes, happen more frequently, often not cohesive, often bug fixes instead of new functionality.



Individual pieces of functionality, happen often, most important piece of functionality at the time, relatively low customer absorption costs.

Major releases are tragedies



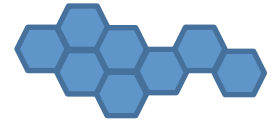
- They are major because we haven't figured out how to deliver functionality less tumultuously
- They conform to our limitations rather than drawing on our strengths
- The software behind our products is usually so bad that we have to develop and deliver a lot to deliver anything

Those that are swimming naked, want the tide to go out as infrequently as possible.

Minor (point) releases are for triage



- Point releases are significant bug fixes or features that must be delivered before the next major release
- A point release is a wide, thin collection of functionality that is layered onto an existing implementation
- It can have a large impact on the software, but its impact on the customer should be small
- Has to be worked into the previous releases, as well as upcoming major releases, further complicating the work to be done.



What they are

- Releases new functionality as soon as it is ready (Done)
- Iterative, incremental development processes generate these as one or more increments, in one to several months
- A major release can be a simple declaration of a grouping of functional releases if that is what your customers expect.

What they require

- Software must be structured so functions can be built or changed in isolation
- Customers must be able to implement functionality in just one area, without complications to other areas that use the same product.
- The support organization must understand which function a customer is utilizing

Requirements for managing multiple releases simultaneously

Technical requirements

- Ability to manage multiple sets of code at once
- Ability to assess releasability
- Ability to keep builds from each branch releasable
- Ability to readily merge code branches into trunk

Customer requirements

- Customer must be able to implement them without affecting the rest of the major release code
- The release organization must be able to discretely identify each Functional release (1.1.4)
- The customer and support organization must be able to identify the release numbering for the any support issues

Weave around development constraints

Release	Description
Organization of software	Software may be organized so that there is a minimum part that must be changed, regardless. Architecture, common components, coupling and cohesion all contribute to this constraint. UI frameworks or data abstraction layers can also be constraints.
Testing Environments	Incomplete, manual tests will slow down any desired release.
Development environments	How many unique development environments can be sustained at one time? What are code branch constraints?
Size of development organization	Per person, a large development organization is much less productive than a small one.
Development velocity	The ability of the developers to turn requirements into working functionality is low and difficult to improve.

Is faster better?



10 MINS

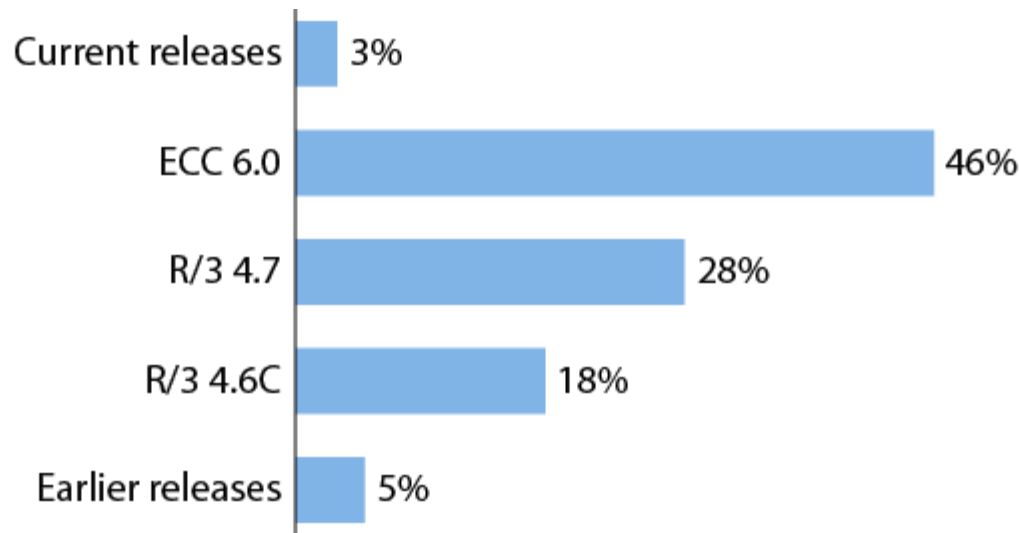
- Amazingly, your development organization can suddenly work on multiple releases at once. They can even get small, point releases out the door in one month
- Does this help you?
- Considerations
- How many releases can your organization support at once?
- What is the maximum that your customers can readily absorb?
- What does this all cost?

Take a lesson from SAP, absorption is important for value delivery

FORRESTER

June 2010 “**Application Upgrades: How To Make Upgrade Decisions When Business Value Proves Elusive**”

Half Of SAP Customers Have Fallen Behind On Upgrades



Base: 44,615 productive systems

Source: SAP SAPHIRE NOW keynote by SAP Supervisory Board Chairman and Co-founder Hasso Plattner, May 19, 2010

Weave around customer constraints

Release	Description
Regression	A release may not be less capable than prior releases.
Release environments	How many releases can be simultaneously supported and sustained?
Release Service Levels	How many releases back must be maintained by licenses?
Customer Absorption	How often can the customer base absorb new releases? How often do they want to?
Release Scope	Must the release be for all platforms and international markets, or can it be for subsets?
Service Level Agreements	How often must releases be made, what is their minimum content, and is distribution strategy dictated by license?

Microsoft Office release strategy

- Microsoft Office product family built on an object model that provides common services
- Products must be updated or enhanced all at once, which makes it hard to do functional releases
- Microsoft Office is developed using waterfall, even though much of Microsoft uses Scrum



Agenda

- Agility is necessary
- Empiricism and transparency are necessary for Agility
- Scrum is a tool you use to become Agile
- Done is not always transparent
- Agile releases
- **ALM provides the tooling for Agility**

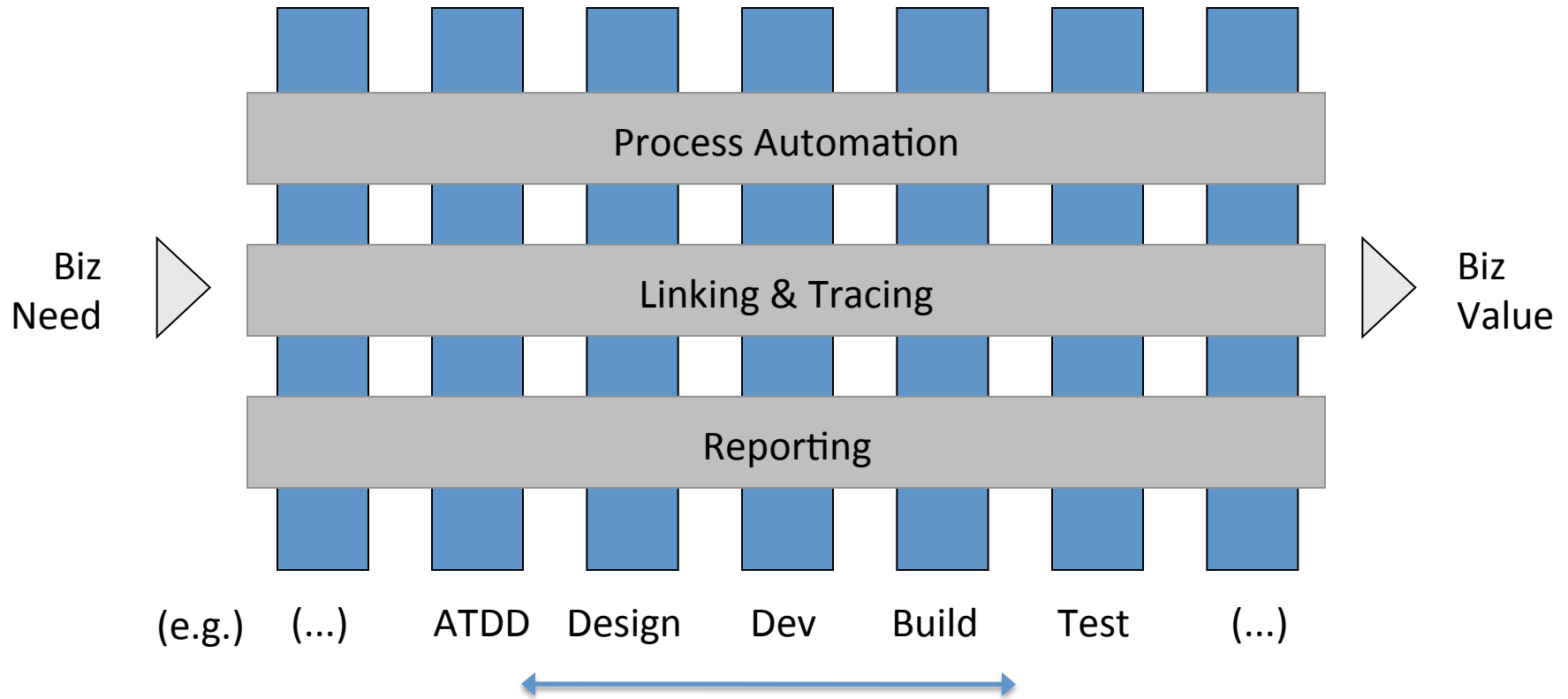
What Is ALM?

Application Life-cycle Management (*n.*) is the coordination of development life-cycle activities, including requirements, modeling, development, build, and testing, through:

1. Enforcement of processes that span these activities;
2. Management of relationships between development artifacts used or produced by these activities; and
3. Reporting on progress of the development effort as a whole.

Note: ALM is a set of practices before it is a tool. ALM tools increase the efficiency and effectiveness of ALM practices.

(No, really, what is ALM?)



(Develop code, tests, documentation)
(Use modern Test First Development practices)

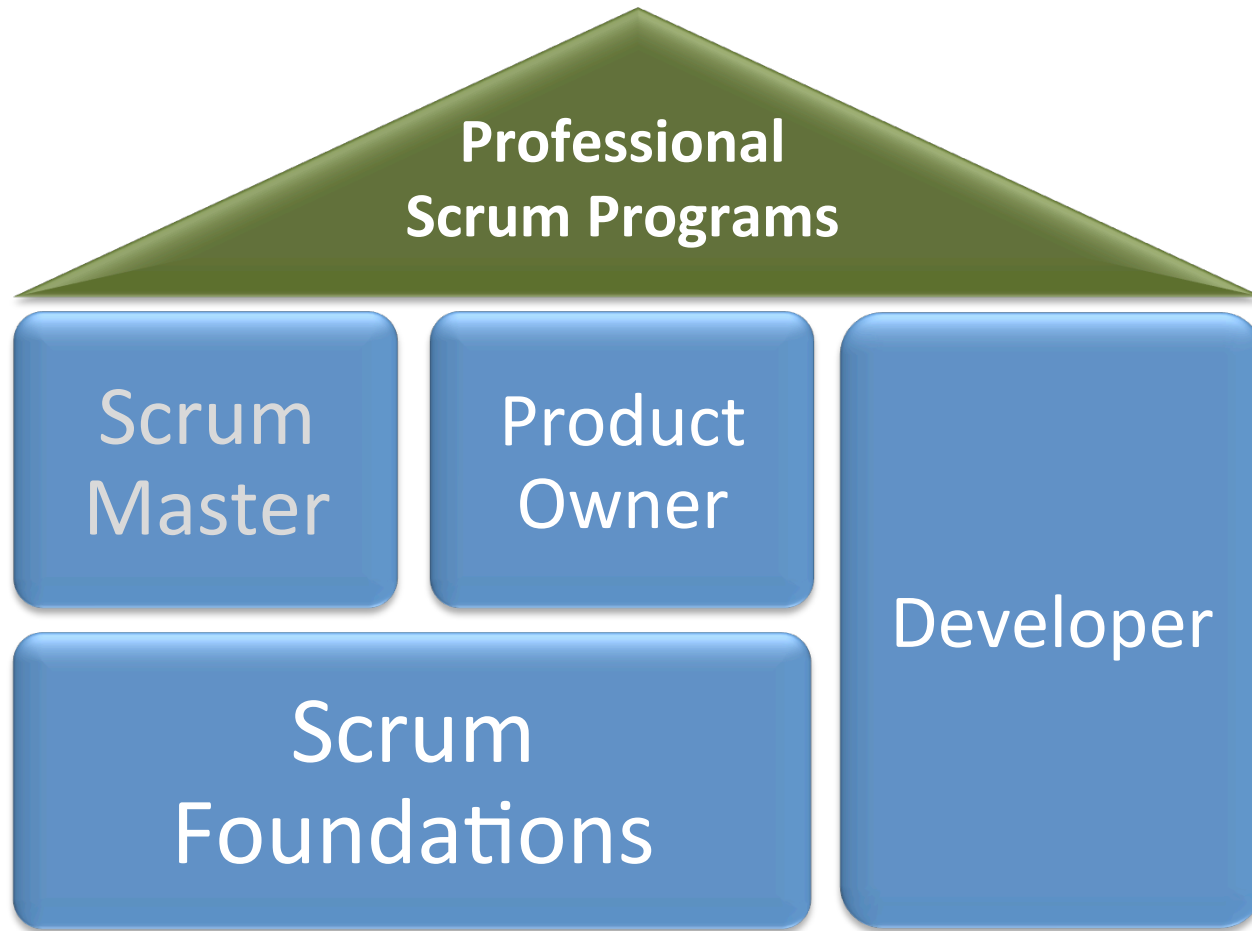
What Do Scrum And ALM Have In Common?

Scrum and ALM improve the chances that all of the work a development organization does will actually add up to working software that meets business needs and deliver business value.

Scrum.org programs

- Every Program consists of:
 - Partners
 - Trainers and coaches
 - Body of knowledge
 - Assessments
- Every Program is monitored and its quality assured by Scrum.org,

Filling In Scrum's Holes



Professional Scrum Developer

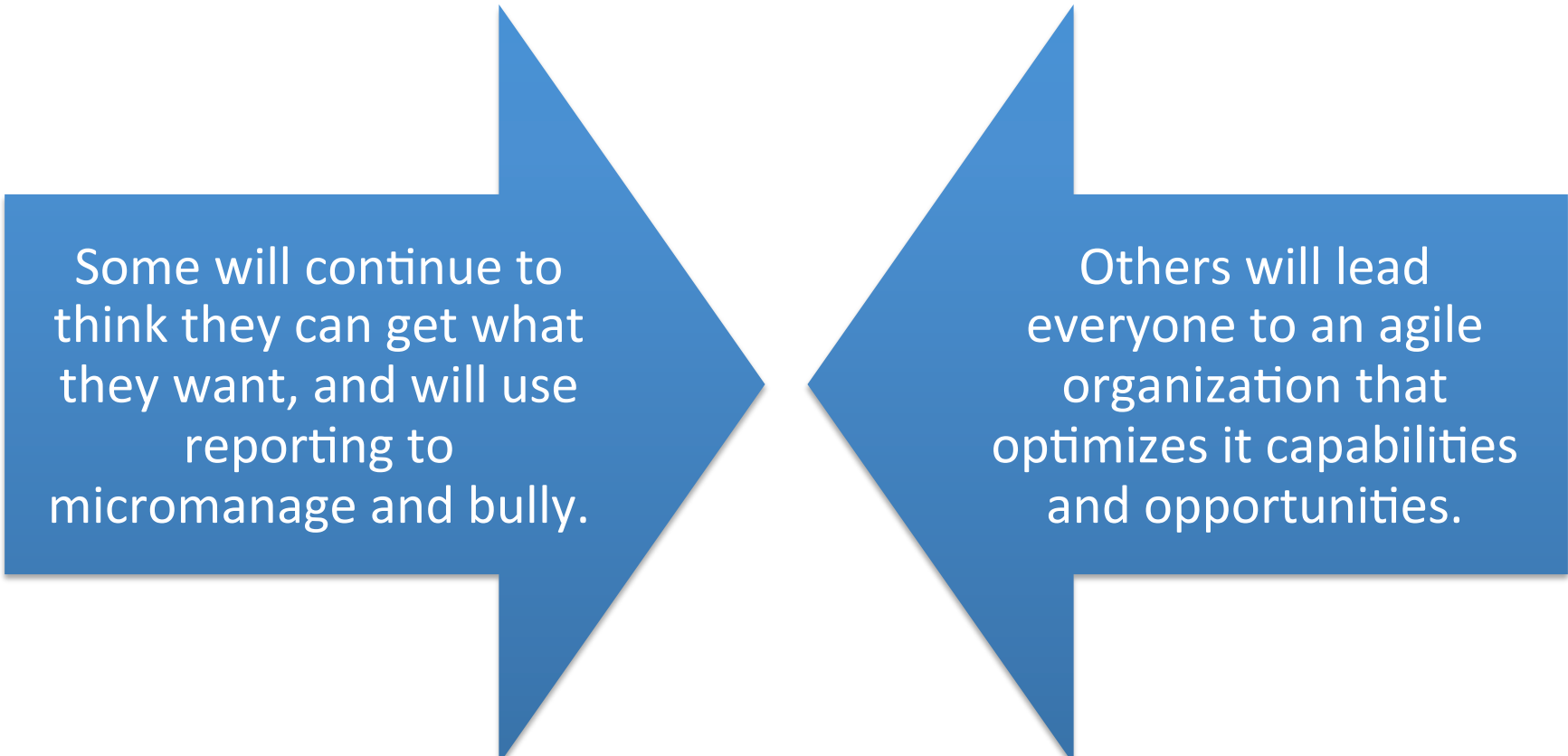
The official Microsoft training for Visual Studio 2010

Teaches VS2010 and best practices, all within the Scrum framework. The Microsoft developed Scrum Process Template for VS2010 is the most faithful to Scrum process template available.

Recognizes that software development today is different than it was 10 years ago. Goes beyond traditional “tool-only” courses to also teach best practices, and how to develop on a x-functional team using Scrum.



ALM Is As Good The Culture



Some will continue to think they can get what they want, and will use reporting to micromanage and bully.

Others will lead everyone to an agile organization that optimizes its capabilities and opportunities.

The Quiet Revolution

- More organizations are using Agile than waterfall, and 84% of those using Agile are using Scrum.
- Microsoft's ALM with Scrum contains the tools, processes and techniques necessary for Agility.
- Scrum.org's Professional Developer Program helps people become Agile.
- The success stories from Agile are compelling.

Next Steps

Stay Connected



Subscribe to our RSS feed to receive the latest updates on scrum.org programs, training, and certifications. <http://www.scrum.org/news/rss.xml>



Follow us on Twitter, our username is [@scrumdotorg](https://twitter.com/scrumdotorg)

Engage



Join the Yahoo! discussion group where you can tap into the collective knowledge of Scrum users around the world.

<http://groups.yahoo.com/group/scrumdevelopment/>



If you achieve PSPO certification you are eligible to join the LinkedIn group to continue the conversations with the people you've met in this class, and start new ones with other PSPOs.

Thank you!

Ken Schwaber

ken.schwaber@scrum.org

@kschwaber

kenschwaber.wordpress.com

www.scrum.org

Agenda

- Agility: what is it?
- Scrum is a tool to become Agile
- Start Small
- Agile Releases
- Quick Scrum
- **Path to Agility**

How to use Scrum

- Bottom-up with permission and support
- Critical projects
- Isolated areas of organization
- Organization wide adoption of Agility

Agility Requires Organizational Change

Culture

The set of shared attitudes, values, goals, and practices that characterizes an organization.

“The way we do things here.”

- An organization’s **culture** is finely tuned to produce its current problems.
- Agility is an entirely new state.
- The culture must be changed in order to achieve Agility.
- Organizational change is a *difficult* multi-step process that requires **leadership**.

Types of Change Scrum Requires

- Empirical management replaces predictive management.
- Transparency is value neutral.
- Authority moves down the organization.
- More attention and hard choices are required.

Engagement model: The Path to Agility

- We have developed an engagement model to guide you to achieve Agility.
- Based on:
 - Twenty years of experience.
 - “CxO Playbook” first used in 2005 and updated after engagements.
 - In collaboration with John Kotter’s*, adoption of his model for organizational change.

* Harvard professor internationally recognized as leading expert on organizational change.

Wrapped in an organizational change program



CXO Playbook

For Adopting Scrum & Achieving Agility
Version 0.9

Eight stage process for creating culture change

1. Establish a sense of urgency (what will happen if we stay the same?).

Eight stage process for creating culture change

2. Create a guiding coalition (group is a Team with adequate power and influence).

Eight stage process for creating culture change

3. Develop a vision and strategy (what will it be like when we are changed?).

Eight stage process for creating culture change

4. Communicate the change vision (use every possible vehicle and ensure leaders exemplify vision).

Eight stage process for creating culture change

5. Empower broad-based action (eliminate obstacles and encourage risk-taking).

Eight stage process for creating culture change

6. Generate short-term wins (immediate, visible success that is celebrated).

Eight stage process for creating culture change

7. Consolidate gains and produce more change (promote advocates and leaders).

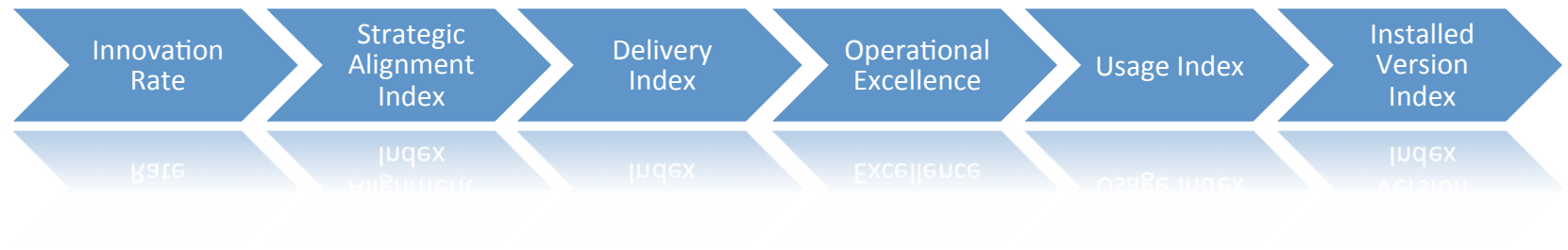
Eight stage process for creating culture change

8. Anchor new approaches into the culture
(change disappears if not continually fostered).

Path to Agility: Playbook

Play	Description
1. Initiation	Set baselines, form leadership team, establish vision and urgency, begin communicating, determine metrics, identify pilots
2. Pilots	Create wins, communicate, remove impediments, start change, gather initial metrics and dashboards, provide initial training.
3. Organizational Expansion	Expand usage of Scrum, inspect metrics and adapt accordingly, remove impediments to agility for organization and customers, remediate products and systems, communication internally and externally, more training and coaching. Create anchor points.
4. Achieve Impact	Restructure relationships, processes, management, organizational structure, career paths, cultural values, and everything required to optimize productivity and remove waste. Start managing results of change.
5. Anchor	Review all practices, structures, and processes in organization. Anchor change into them so new culture is created.

Six simple KPIs will give you a great start



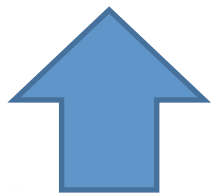
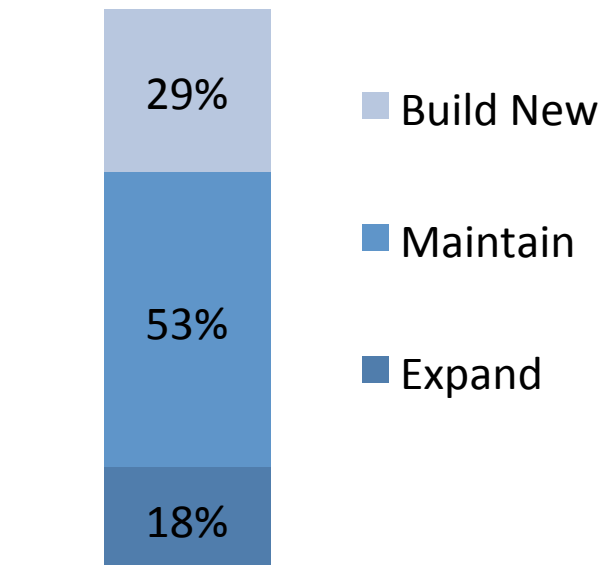
Innovation Rate

Building new vs. sustaining existing

What percentage of your product budget is spent on building new functionality vs. maintaining the existing?

- Cross-industry average is 29%
- Customers expect the creation of new value

2010 IT Budgets

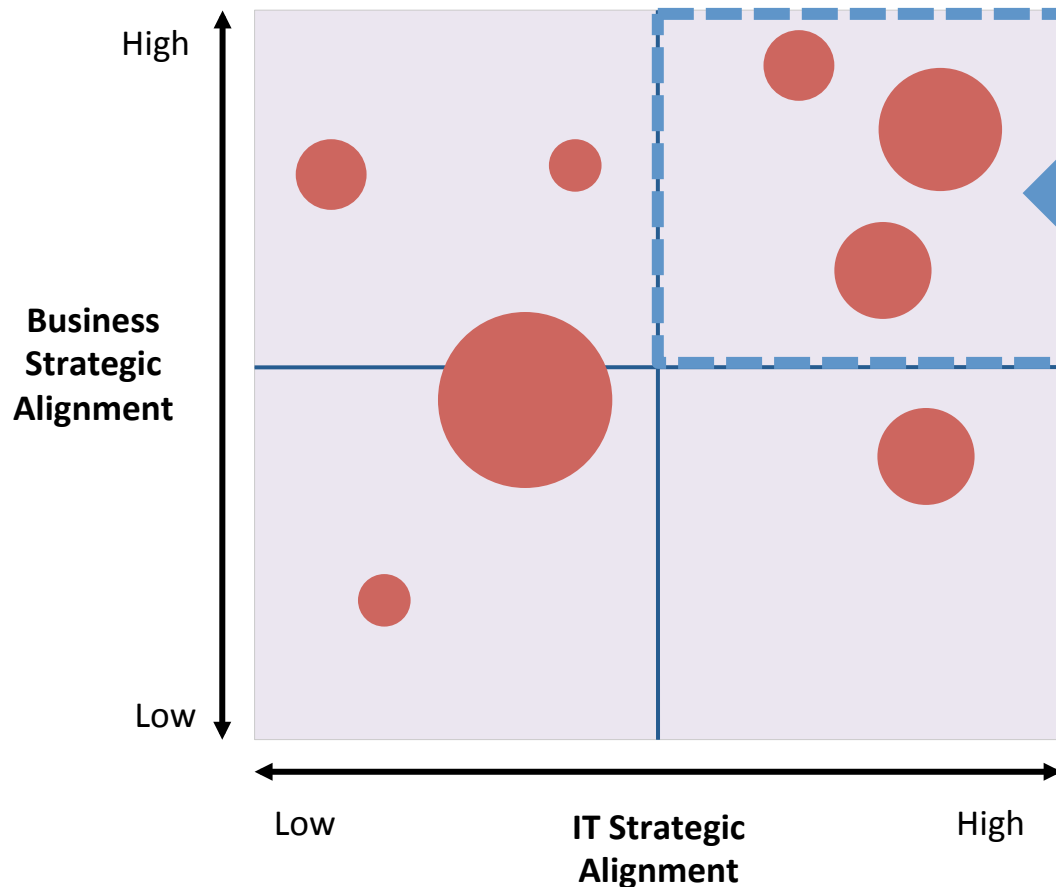


Generally, higher is better

Strategic Alignment Index

Software to support strategic vision and IT strategy

Strategic Alignment Map

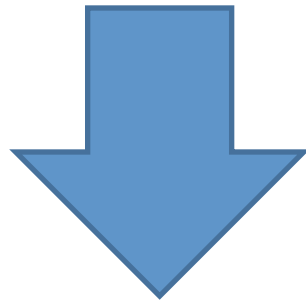


Highest benefits are most likely realized when building these products or features

Size of bubble = TCO

Prioritize projects and features that align with both business goals, and IT strategy.

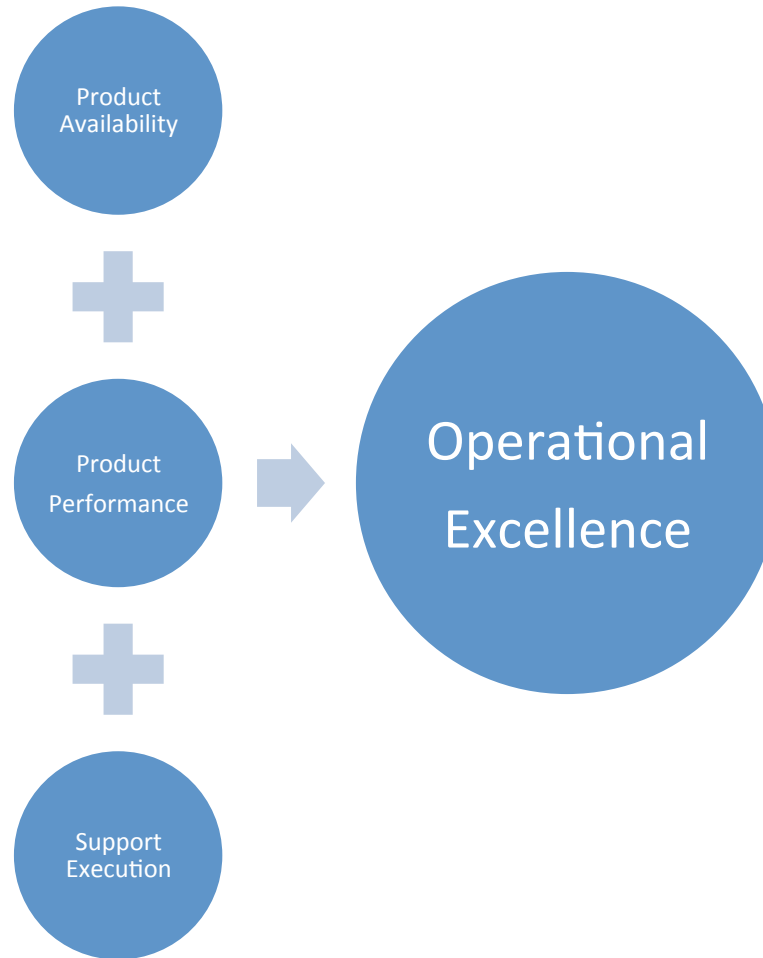
Percent of time team spends
working on non-product stuff
(e.g. production issues)



The lower the better

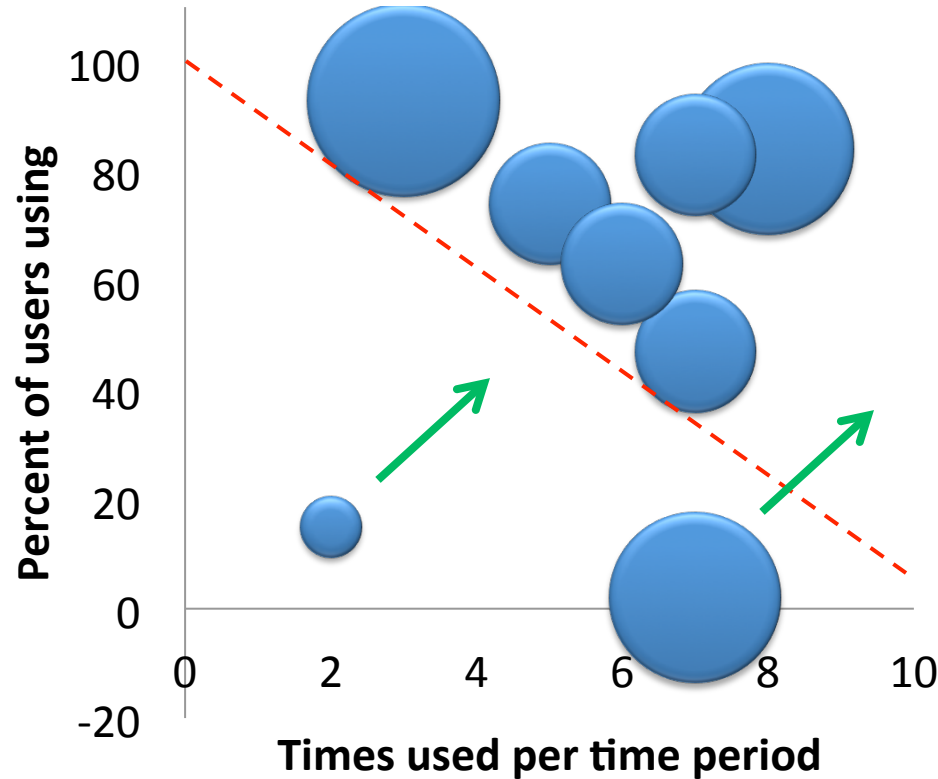
Operational Excellence

Customer access, usability, and support



Usage Index

Do people actually use the functionality you build?



Size of bubble = Time spent using

Build things that people will actually use.

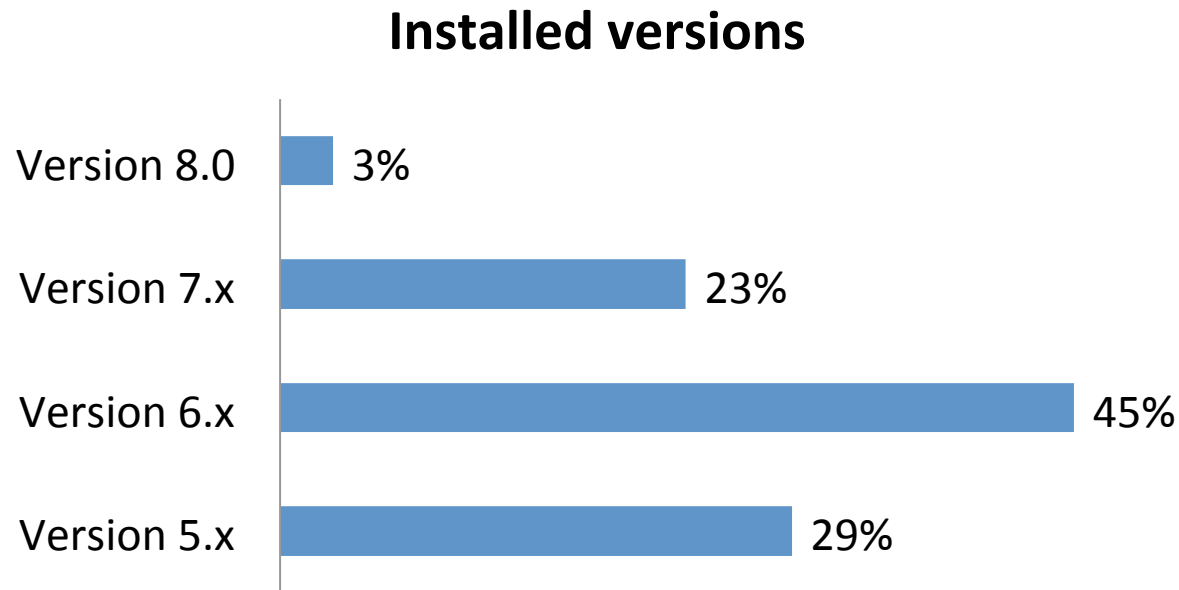
If they aren't, try to figure out why and drive more usage. If you can't, get rid of the feature.

Higher tends to be better

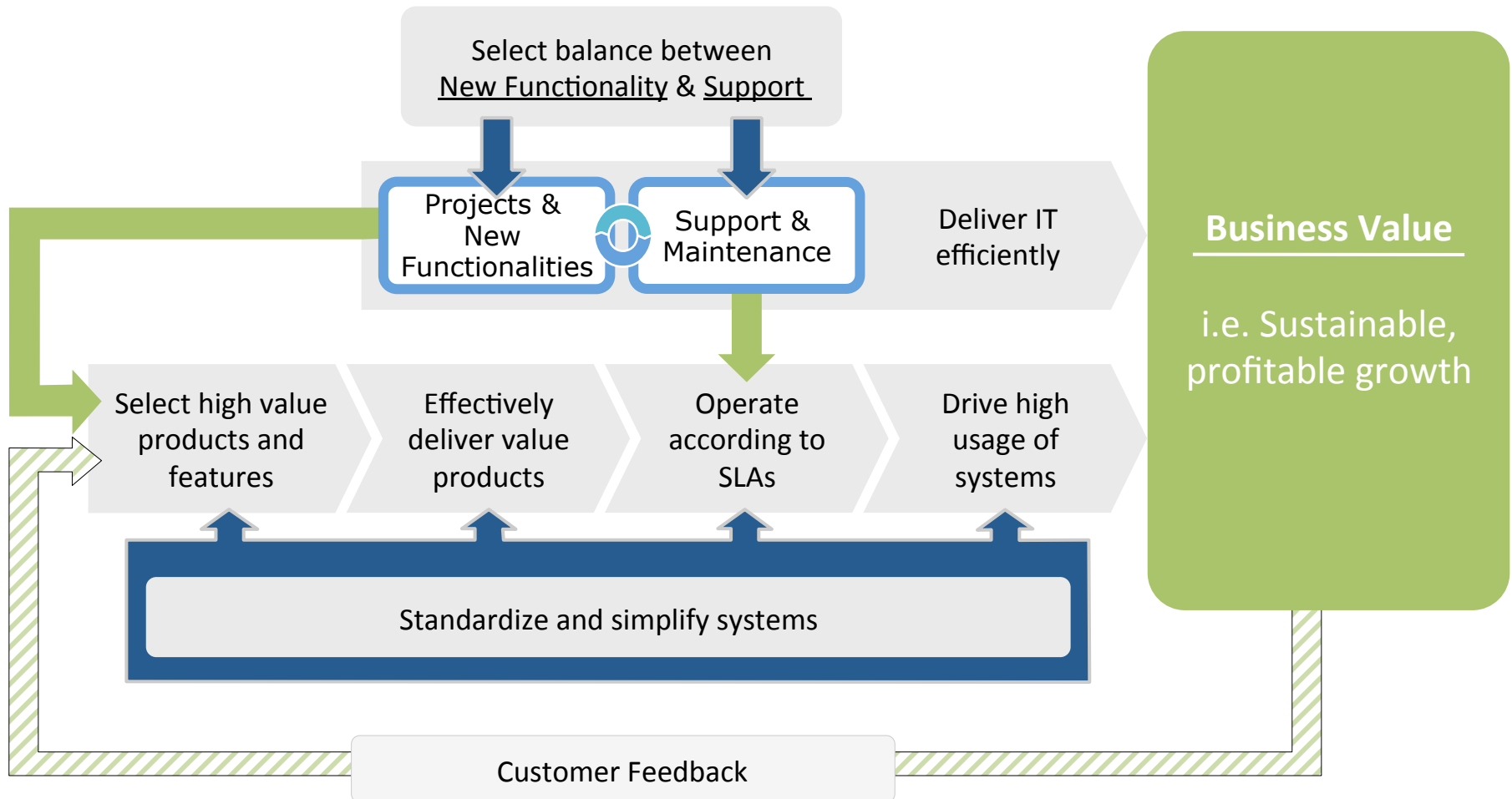
Installed Version Index

Monitoring product absorption

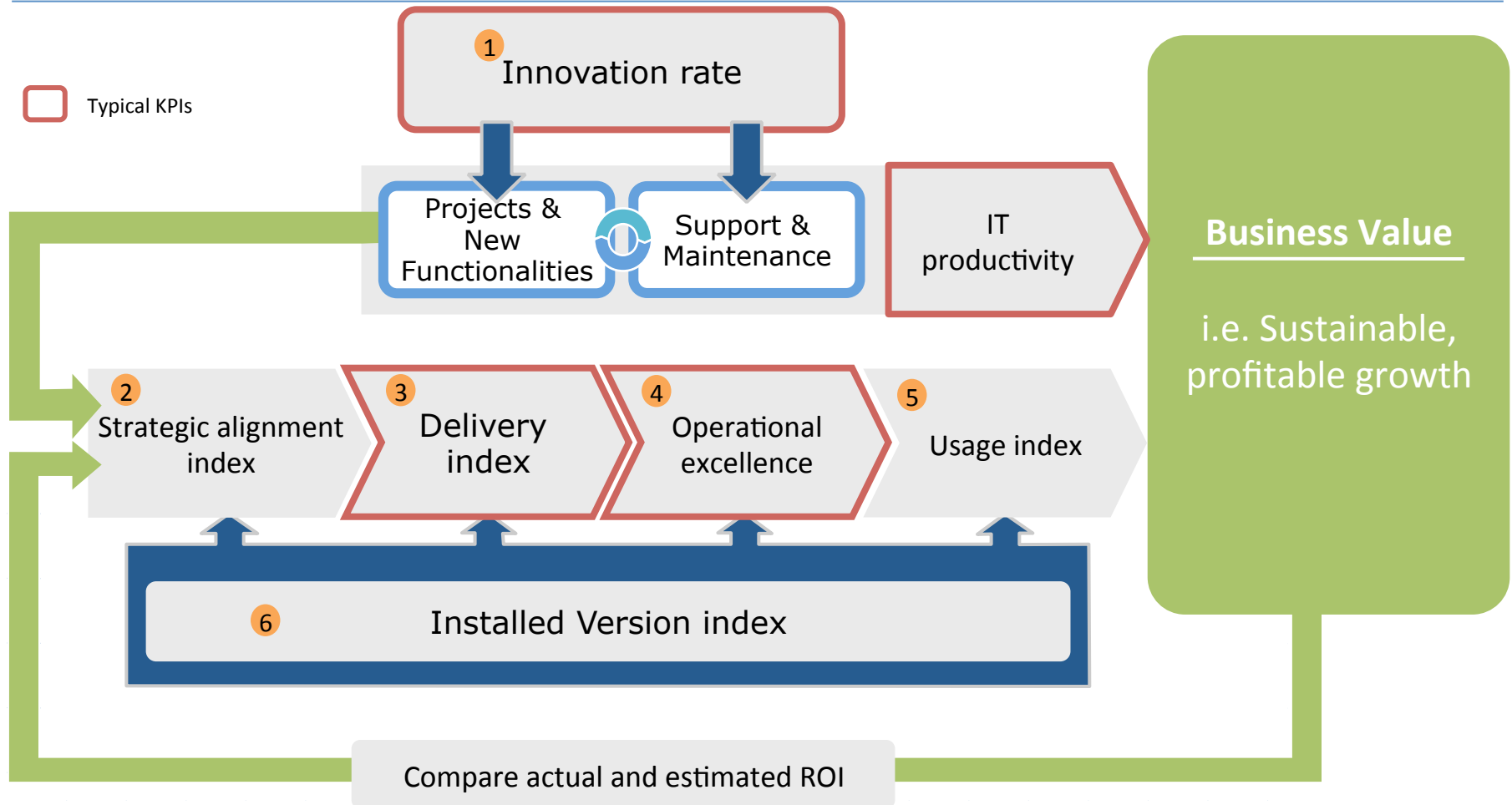
What percentage of your customers are on your latest (n) release?



Product Owners need to think about much more than delivering on time and on budget



Specific KPIs can be used at each point to determine how well a PO delivers business value



Using KPIs provides a high-level proxy of actual value delivered by software products

